



INSTITUTE OF INFORMATION TECHNOLOGY
UNIVERSITY OF DHAKA



TERRA-AUDIT

*– AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land
Management Carbon Credits*

Submitted As

Technical Report

SE801 Project Final Technical Report

Submitted By

Kazi Nahid

BSSE Roll: 1437

Supervised By

Dr. Emon Kumar Dey

Professor

Institute of Information Technology

University of Dhaka

Submission Date

Oct 10, 2026

Letter of Transmittal

October 10, 2026

BSSE 4th Year Exam Committee

Institute of Information Technology

University of Dhaka

Subject: Final Report Submission — Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits

Dear Sir,

I am submitting the technical report for my Software Project Lab 3 project, Terra-Audit: AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits. This report covers the project's requirements, software design, implementation, interface design, testing, user guidance, and the limitations of the current system. I welcome your feedback and suggestions for further improvement.

Thank you for your consideration.

Sincerely,

Kazi Nahid

BSSE-1437

Institute of Information Technology

University of Dhaka

October 10, 2026

Supervisor's Signature

Abstract

Alternate Wetting and Drying (AWD) irrigation and improved agricultural land management (ALM) can reduce greenhouse-gas emissions, but the monitoring, reporting and verification (MRV) evidence behind carbon claims is usually scattered across field logs, spreadsheets and disconnected calculations. Terra-Audit is an evidence-centred digital MRV platform that prepares agricultural carbon projects for audit. It implements two scoped accounting pathways—Verra VM0051 v1.1 for AWD rice and VM0042 v2.2 for cropland ALM with a restricted VMD0054 v1.1 leakage adapter—on a Next.js frontend, a FastAPI backend and a durable database-backed worker. The platform provides organization and project access control, structured evidence records, Sentinel-1 SAR analytics through Google Earth Engine, methodology bundles with deterministic readiness checks, immutable versioned calculation snapshots, version-pinned internal review and audit-oriented exports. Random Forest/XGBoost detectors, managed crop models and bounded generative-AI explanations are assistive; deterministic server code remains the only authority for accounting values and readiness states.

On 7 October 2026 the complete backend suite passed (177 tests, 0 failures), together with the frontend tests, production build, TypeScript check and lint (0 errors). Independent AWD field validation, complete methodology coverage and external VVB/Verra verification remain outside the delivered scope and are discussed as limitations.

Keywords: digital MRV, AWD rice, VM0051, VM0042, carbon accounting, evidence provenance, satellite monitoring, readiness, internal review.

Table of Contents

1. Introduction.....	1
1.1 Project Title.....	1
1.2 Problem Statement and Motivation.....	1
1.3 Objectives.....	1
1.4 Scope.....	2
1.5 Deliverables.....	3
1.6 Project Timeline.....	3
1.7 Organization of the Report.....	4
2. Background and Related Work.....	4
2.1 AWD Irrigation and Rice Methane.....	4
2.2 Satellite Monitoring of Rice Fields.....	5
2.3 Carbon Methodologies and Digital MRV.....	5
2.4 Gap Addressed by Terra-Audit.....	5
3. Requirements Analysis.....	5
3.1 Requirements Inception.....	5
3.2 Functional Requirements.....	6
3.3 Non-functional Requirements.....	7
3.4 Stakeholders and Viewpoints.....	8
3.5 Quality Function Deployment.....	10
3.6 Usage Scenarios and User Stories.....	10
4. Scenario-Based Modeling.....	10
4.1 Use Case Diagrams.....	10
4.1.1 Use Case Level 0: System Context.....	11
4.1.2 Use Case Level 1: Modules of Terra-Audit.....	11
4.1.3 Use Case Level 2: Carbon Calculation.....	13
4.1.4 Use Case Descriptions.....	13
4.2 Activity Diagrams.....	15
4.2.1 Cropland ALM Detailed Activity Flow.....	16
5. Data-Based Modeling.....	17
5.1 Entity-Relationship Diagram.....	17
5.1.1 Relationship Summary.....	19
5.2 Database Schema.....	19
5.3 Dataset Description (AI-Assisted Detection).....	20
6. Class-Based Modeling.....	21
6.1 Analysis Classes.....	21
6.2 Class Cards (CRC Model).....	21
6.3 Design Class Diagram.....	22
6.3.1 Design Class to Implementation Mapping.....	23
7. Architectural Design.....	24
7.1 Architectural Context Diagram.....	24
7.2 Defining Archetypes.....	25
7.3 Top-Level Components.....	26

7.4 Mapping Requirements to Software Architecture.....	27
7.5 Technology Stack.....	28
7.6 Design Evolution and Alternatives.....	28
8. Component-Level Design.....	28
8.1 Persistent Data Sources and Interfaces.....	28
8.1.1 Context-Level Data Flow (DFD Level 0).....	29
8.1.2 Level-1 Data Flow (DFD Level 1).....	29
8.2 API Design.....	30
8.3 Behavioral Design.....	31
8.3.1 Behavioral Event Table.....	31
8.3.2 State Diagrams.....	31
8.3.3 Sequence Diagrams.....	33
8.4 Deployment Design.....	35
9. AI Engineering Design.....	36
9.1 Classical AWD ML Pipeline.....	36
9.2 Model Selection and Evaluation.....	37
9.3 Managed Crop Models.....	37
9.4 Generative AI Explanation Architecture.....	37
9.5 Prompt and Context Design.....	37
9.6 AI Authority Boundary and Evaluation Limitations.....	38
10. Interface Design.....	38
10.1 Interface Objects and Actions.....	39
10.2 Primary User Workflows.....	39
10.3 UI Events and State Transitions.....	39
10.4 Responsive, Validation and Accessibility Design.....	40
10.5 Interface States as Seen by the User.....	40
11. Implementation.....	44
11.1 Final Feature Status.....	44
11.2 Worked Calculation Checks.....	44
12. Testing.....	44
12.1 Testing Objectives.....	44
12.2 Testing Approach, Strategy and Pass/Fail Criteria.....	45
12.2.1 API Endpoint Verification.....	45
12.3 Detailed Test Cases.....	46
12.4 Verification Results.....	47
12.5 Risks and Contingencies.....	48
12.6 Requirements Traceability Matrix.....	49
13. User Manual, Deployment and Repository.....	50
13.1 End-User Workflows.....	50
13.2 Export and Troubleshooting.....	51
13.3 Local Installation and Development Setup.....	51
13.4 Deployment and Public Repository.....	52
14. Evaluation, Limitations and Conclusion.....	52

14.1 Final Evaluation.....	52
14.2 Methodology and Accounting Limitations.....	53
14.3 Scientific Validation Limitations.....	53
14.4 Provenance and Reproducibility Limitations.....	53
14.5 Security and Operational Limitations.....	53
14.6 Generative-AI Limitations.....	54
14.7 Prioritized Future Work.....	54
14.8 Conclusion.....	54
References.....	54

List of Figures and Tables

Figures

- Fig 1 Project timeline (Gantt chart) — p. 4
- Fig 2 Use Case Level 0 — Terra-Audit as a single system with its actors and external systems — p. 10
- Fig 3 Use Case Level 1 — the ten user-goal modules and their actors — p. 10
- Fig 4 Use Case Level 2 — Carbon Calculation (1.4) with «include» and «extend» relationships — p. 11
- Fig 5 Overall activity diagram with swimlanes for the project user, the Terra-Audit server and the durable worker — p. 14
- Fig 6 Activity diagram — methodology readiness and internal review with swimlanes — p. 15
- Fig 7 Entity-relationship diagram (crow's-foot notation) of the core persistent entities — p. 16
- Fig 8 Entity-relationship diagram of the calculation, readiness and internal-review tables — p. 17
- Fig 9 CRC cards for the nine core design classes — p. 20
- Fig 10 Design class diagram: principal classes and modules, key operations and dependencies — p. 21
- Fig 11 Architectural context diagram (ACD) of Terra-Audit — p. 22
- Fig 12 Layered architecture archetype instantiated for Terra-Audit — p. 23
- Fig 13 Pipe-and-filter archetype: the Sentinel-1 signal-processing pipeline of the rice AWD pathway — p. 23
- Fig 14 Top-level component diagram of Terra-Audit — p. 24
- Fig 15 Level-0 data-flow diagram — p. 27
- Fig 16 Level-1 data-flow diagram: processes, data stores and external flows — p. 28
- Fig 17 State diagram — durable background job lifecycle (src/jobs/queue.py) — p. 30
- Fig 18 State diagrams — (a) calculation version lifecycle and (b) data-quality issue lifecycle — p. 30
- Fig 19 State diagram — internal review submission lifecycle (src/projects/reviews.py) — p. 31
- Fig 20 Sequence diagram — calculation commit from authenticated request to immutable version — p. 31
- Fig 21 Sequence diagram — Sentinel-1 signal run through the durable job queue — p. 32
- Fig 22 Sequence diagram — bounded AI explanation with server-side validation — p. 33
- Fig 23 UML deployment diagram of Terra-Audit — p. 34
- Fig 24 AI authority and data flow — p. 36
- Fig 25 Application shell and project navigation — p.
- Fig 26 Monitoring batch state: one running batch with two pending child jobs; no data-quality issues — p. 38
- Fig 27 Rice calculation readiness and inputs — p. 39
- Fig 28 Review-related states — p. 40
- Fig 29 AI workspace — crop-model controls — p. 40

Tables

- Table 1 Functional requirements and priorities — p. 5
- Table 2 Non-functional requirements and verification targets — p. 7
- Table 3 Stakeholders and viewpoints — p. 8
- Table 4 Use-case inventory and actor mapping — p. 11
- Table 5 Detailed use-case specifications — p. 11
- Table 6 Persistent database schema by domain — p. 18
- Table 7 Analysis-class identification from scenario nouns — p. 19
- Table 8 Design classes and their implementing modules — p. 21
- Table 9 Component responsibilities — p. 24
- Table 10 Mapping functional requirements to architectural elements — p. 25
- Table 11 Technology stack — p. 25
- Table 12 REST API summary by router group — p. 28
- Table 13 Behavioral event table — p. 29
- Table 14 User-interface objects and actions — p. 36
- Table 15 UI events and resulting state changes — p. 37
- Table 16 Direct API endpoint verification — p. 42
- Table 17 Detailed test cases with outcomes — p. 43
- Table 18 Verification results (7 October 2026, commit bc402f2) — p. 44
- Table 19 Functional requirements traceability matrix — p. 45
- Table 20 End-user workflows — p. 47
- Table 21 Export options and troubleshooting — p. 47

1. Introduction

1.1 Project Title

Terra-Audit — AI-Assisted Digital MRV Platform for AWD Rice Irrigation and Agricultural Land Management Carbon Credits.

1.2 Problem Statement and Motivation

Rice paddies are a major source of anthropogenic methane, and Alternate Wetting and Drying (AWD) irrigation is a recognized mitigation practice, but evidence collection and carbon accounting remain difficult to scale and audit. Farmer irrigation logs can be incomplete or difficult to verify, continuous field instrumentation can be costly across many smallholder fields, and spreadsheet-based accounting often separates the reported result from the evidence, assumptions, methodology version, and calculation history that produced it. A parallel problem exists for improved agricultural land management (ALM), where practice changes, production data, soil organic carbon (SOC) measurements, leakage considerations, and uncertainty must be assembled from different evidence sources and retained consistently across verification periods. Terra-Audit addresses these problems as an evidence-first digital MRV platform with two scoped accounting pathways. For rice fields, Sentinel-1 SAR observations from Google Earth Engine support analysis of flooding and drydown behavior; for cropland ALM fields, structured practice, soil, laboratory, production, livestock, and leakage evidence supports the accounting workflow. The platform combines deterministic carbon calculations, methodology-bundle and readiness checks, versioned calculation snapshots, internal review, durable background processing, and exportable evidence records. Satellite and AI outputs are treated as supporting analytical evidence rather than as substitutes for independent field validation or external VVB/Verra verification.

1.3 Objectives

- Use freely available Sentinel-1 SAR observations to provide scalable supporting evidence of rice-field flooding and drydown behavior while clearly separating satellite-derived analytics from independently validated AWD ground truth.
- Provide a parallel evidence-driven cropland ALM pathway based on the project's scoped implementation of Verra VM0042 v2.2, using structured practice, production, livestock, soil-sampling, laboratory, custody, and reviewed SOC evidence rather than rice satellite signals.
- Implement transparent deterministic carbon accounting for the supported subsets of Verra VM0051 v1.1 (rice AWD, QA3/default-factor pathway) and VM0042 v2.2 (cropland ALM), exposing important intermediate values and methodology assumptions for review.
- Represent the evidence lifecycle explicitly, including crop seasons and corrections, observations, practice events, attachments, production records, soil sampling, laboratory results, chain-of-custody records, reviewed SOC inputs, and supported integrated crop-livestock evidence.
- Integrate methodology bundles, corrections, applicability, readiness checks, and restricted VMD0054 v1.1 leakage logic so unsupported, incomplete, or reviewer-dependent requirements are disclosed instead of being silently treated as satisfied or zero.

- Preserve calculation history through versioned calculation records and frozen evidence/input/result snapshots, and support internal review of eligible calculation versions without presenting that workflow as external VVB validation or registry verification.
- Support AI-assisted analysis while keeping deterministic accounting authoritative: classical Random Forest/XGBoost models and managed crop models remain experimental, while generative AI is limited to bundle-scoped draft explanations that are validated by server-side rules and require human review.
- Provide an authenticated multi-user web workflow with organization/project access control, durable background jobs, calculation history, review and export facilities, and sufficient traceability for audit preparation while clearly disclosing current security, provenance, and methodology limitations.

1.4 Scope

In scope:

- Authenticated account, organization, project, farm, and field workflows, including field registration from supported geometry inputs and server-side organization scoping.
- Two registered accounting pathways — rice_awd and cropland_alm_vm0042 — with field type treated as an immutable routing attribute after registration.
- Rice signal analytics using Sentinel-1 VV/VH observations, compatible caching, durable signal jobs, smoothing, threshold-based AWD/phenology logic, and optional experimental ML detectors.
- Structured cropland evidence for VM0042, including crop seasons, practice schedules, production records, soil sampling plans/strata/samples, laboratory results, custody history, reviewed SOC mapping, and supported non-SOC emission inputs.
- Baseline/project livestock schedules for the supported integrated crop-livestock scope.
- Supported livestock emission terms, including selected enteric-fermentation and pasture/range/paddock manure sources within the scoped ALM engine.
- Project-, field-, bundle-, and reporting-period-scoped leakage assessment using the implemented restricted VMD0054 v1.1 adapter and structured production evidence.
- Versioned calculation history with frozen inputs, results, readiness state, methodology-bundle context, and supersession metadata; eligible versions can enter the internal review workflow.
- Methodology document/bundle management, corrections, project applicability, deterministic readiness evaluation, findings, and reviewer workflow.
- Deterministic scoped accounting for VM0051 v1.1 rice and VM0042 v2.2 ALM, including the implemented uncertainty, leakage, SOC, and selected non-SOC terms.
- Classical AWD ML, managed crop-model experimentation, and generative draft explanations, all explicitly separated from authoritative accounting and external verification decisions.
- Supported export paths include legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF; the application also provides private evidence attachments, calculation history, and operational/admin views for jobs, methodology sources, providers, and worker state.

- Primary research focus remains Bangladesh/South Asia for the rice pathway. The application supports SQLite for local/test use and PostgreSQL plus local or private S3-compatible storage configurations for hosted operation.

Out of scope (this phase):

- Independent field-level AWD ground-truth validation and continuous on-field sensor integration are not completed in the current project; the satellite detector remains an experimental/supporting evidence source.
- Carbon-accounting pathways other than the implemented VM0051 and VM0042 scopes, and complete implementation of every requirement, tool, safeguard, and procedure associated with those methodologies and the VCS program.
- External validation/verification by a VVB, Verra registry integration, VCU creation/issuance, and any claim that an internally calculated quantity is an issued carbon credit.
- Complete production-security certification and operational assurance, including comprehensive project-level authorization coverage, database foreign-key/row-level-security enforcement, independent penetration testing, verified backup/restore drills, and fully sealed end-to-end provenance for every legacy export path.

1.5 Deliverables

- A Next.js web frontend and FastAPI backend providing authenticated project, field, evidence, monitoring, calculation, review, AI, export, and administration workflows; the legacy Streamlit runtime has been retired.
- A domain-oriented persistence layer supporting SQLite and PostgreSQL, with structured records for accounts, projects, fields, evidence, soil, production, calculations, reviews, methodologies, jobs, AI records, and private attachment/model storage.
- Deterministic scoped accounting engines for VM0051 v1.1 rice and VM0042 v2.2 ALM, including restricted VMD0054 v1.1 leakage integration, methodology/readiness gates, and versioned calculation persistence.
- A durable worker/job system for signal analytics and AI work, together with the Sentinel-1 threshold pipeline, experimental RF/XGBoost and managed crop models, and provider-backed generative explanation infrastructure.
- Structured evidence management, private attachments, calculation snapshots/history, internal review/findings, and workflow-specific export paths: legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF, with legacy-provenance limitations disclosed.
- Methodology source/bundle management, project applicability and readiness checks, calculation history/portfolio views, operational monitoring, and server-validated draft AI explanations.
- Final technical documentation, this user manual, the public GitHub repository and a verified test baseline: on 7 October 2026 (commit bc402f2) the backend suite reported 177 passed and 0 failed, the frontend tests passed (4/4), and the production build, TypeScript check and ESLint (0 errors, 5 warnings) completed successfully.

1.6 Project Timeline

Fig 1 compares the planned schedule from the SRS and midterm report with the actual windows reconstructed from the repository history.

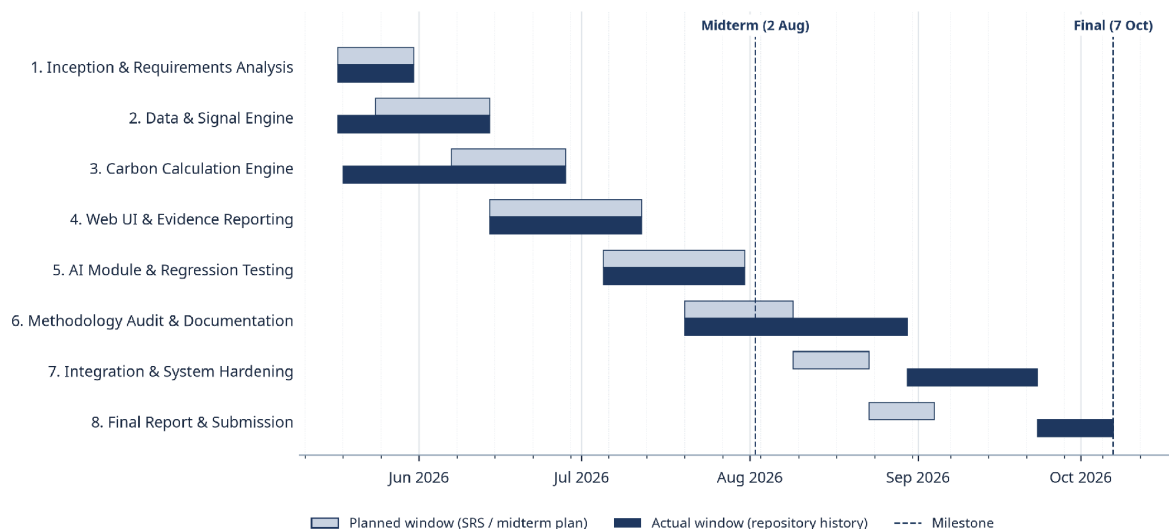


Fig 1: Project timeline (Gantt chart): planned windows versus actual windows, with the midterm (2 Aug 2026) and final verification (7 Oct 2026) milestones.

Phases 1–5 (requirements, signal engine, VM0051 engine, web UI and AI module) finished close to plan and were reported at the 2 August midterm; phases 2 and 3 started early, during inception. Methodology work (phase 6) expanded substantially after the midterm—methodology bundles, corrections, readiness and the VM0042/VMD0054 ALM pathway—so integration, hardening and the final report moved to September and early October. The extension produced the multi-user system, durable jobs, versioned snapshots and internal review described in this report.

1.7 Organization of the Report

Chapter 2 reviews background on AWD, SAR-based rice monitoring and the Verra methodologies. Chapters 3–5 present the requirements, scenario-based models and data-based models. Chapters 6–8 cover class-based modeling, architectural design and component-level design, including the behavioral and deployment models. Chapter 9 describes the AI engineering design, Chapter 10 the interface design, Chapter 11 the final implementation and Chapter 12 testing. Chapter 13 is the user manual with deployment and repository details, and Chapter 14 evaluates the system and concludes the report.

2. Background and Related Work

2.1 AWD Irrigation and Rice Methane

Flooded rice fields emit methane because waterlogged soil creates anaerobic conditions that favour methanogenic microbes. Alternate Wetting and Drying (AWD) lets the field water level fall below the soil surface before re-irrigation, interrupting these conditions and reducing methane emissions while maintaining yield when practised correctly [9]. Verra VM0051 v1.1 quantifies emission reductions from improved rice management using IPCC water-regime, pre-season and organic-amendment scaling factors [1], [4]; its QA3 default-factor approach is applicable only to projects below 60,000 tCO₂e per year.

2.2 Satellite Monitoring of Rice Fields

Synthetic aperture radar (SAR) suits rice monitoring because C-band backscatter responds to standing water and canopy structure and is unaffected by cloud cover. Flooded fields show low VV backscatter because of specular reflection, and backscatter rises as the canopy develops [13], [14]. Sentinel-1 provides free dual-polarized (VV/VH) C-band observations with a 6–12 day revisit [7], [15], and Google Earth Engine offers planetary-scale access to the Sentinel-1 GRD archive [8], [16]. Terra-Audit uses these properties to detect flooding and drydown events, but treats them as supporting evidence because revisit gaps cannot observe every drainage event.

2.3 Carbon Methodologies and Digital MRV

VM0042 v2.2 credits improved agricultural land management through soil organic carbon (SOC) change, non-SOC emission sources and uncertainty deductions, while VMD0054 v1.1 estimates leakage from displaced agricultural production [2], [3]. Conventional MRV for such projects relies on spreadsheets and manually collected documents, which separate the reported number from its evidence, methodology version and calculation history. Digital MRV aims to keep evidence, calculations and review decisions connected and reproducible. Validation and verification remain the responsibility of an accredited VVB under the Verra VCS Program [6].

2.4 Gap Addressed by Terra-Audit

Remote-sensing workflows and carbon-accounting spreadsheets are usually separate tools, so reviewers must reconcile satellite outputs, field evidence and calculations by hand. Terra-Audit combines (i) satellite-derived AWD evidence, (ii) structured ALM evidence, (iii) deterministic methodology engines, (iv) methodology readiness that never treats unsupported requirements as satisfied and (v) immutable, versioned calculation snapshots with internal review in a single multi-user web application.

3. Requirements Analysis

3.1 Requirements Inception

The original SRS established Terra-Audit around the need to make agricultural MRV evidence more traceable and less dependent on disconnected manual records. During implementation, that problem expanded from a single-user rice prototype into a multi-user platform covering project/field management, structured evidence, two scoped carbon-accounting pathways, methodology readiness, durable processing, internal review, and AI-assisted analysis. The final requirements therefore retain the original evidence-first motivation while reflecting the capabilities and constraints of the final implementation.

Requirement elicitation combines the earlier SRS and midterm analysis with implementation evidence from the final system. Carbon-project users need transparent calculations and evidence history; researchers need reproducible satellite/ML workflows; reviewers need explicit readiness and provenance; administrators need secure, observable operations; and external VVB/Verra processes remain outside the application authority boundary. Requirements that cannot be fully automated are represented as partial, unsupported, blocking, or human-review dependent rather than being treated as silently complete.

3.2 Functional Requirements

Table 1: Functional requirements and priorities

ID	Requirement	Priority
FR1	Provide authenticated account access and account-recovery flows; derive the active organization from the authenticated user rather than from a client-selected tenant identifier.	Must
FR2	Create and manage projects, farms, and fields; register each field with a supported geometry and an immutable accounting pathway (rice_awd or cropland_alm_vm0042), and maintain dated project/farm memberships.	Must
FR3	For rice fields, request Sentinel-1 VV/VH observations for a monitoring window, reuse compatible cached or active work where possible, and otherwise execute acquisition/processing through the durable signal-job workflow.	Must
FR4	Process rice time-series with the implemented smoothing, flood/drydown and phenology logic, and retain threshold-based detection as the deterministic reference path for experimental AWD ML models.	Must
FR5	Record crop seasons with correction/version history and associate field observations, practice events, monitoring runs, reviews, data-quality issues, and evidence attachments with the appropriate field/season context.	Must
FR6	For cropland ALM fields, record baseline/project practices, production records, livestock schedules, soil sampling plans, strata, samples, laboratory results, custody events, and reviewed SOC evidence required by the supported calculation scope.	Must
FR7	Calculate the implemented VM0051 v1.1 QA3/default-factor rice result deterministically, including supported CH ₄ scaling, uncertainty and irrigation-change N ₂ O terms, while exposing unsupported methodology elements separately from the numerical calculation.	Must
FR8	Calculate the implemented VM0042 v2.2 ALM subset from frozen evidence, including supported SOC, uncertainty, selected non-SOC emission terms, integrated livestock terms, and restricted VMD0054 v1.1 displacement-leakage logic.	Must
FR9	Support experimental Random Forest/XGBoost AWD detectors and managed crop models without treating model predictions or model-versus-threshold agreement as independent scientific validation.	Should
FR10	Provide bundle-scoped generative AI draft explanations through configured providers, using deterministic server-built context packets and server-side validation; AI output must not control accounting values, readiness, or verification decisions.	Should
FR11	Export the formats implemented for each workflow: legacy verification PDF/JSON/CSV, versioned calculation JSON, and snapshot-backed ALM calculation PDF, while clearly distinguishing snapshot-backed outputs from legacy paths whose historical provenance is not fully frozen.	Must
FR12	Provide field/project history and portfolio views that let authorized users inspect registered fields, current methodology path, calculation versions, latest results, and relevant project context.	Should
FR13	Manage versioned methodology documents, bundles, corrections, project applicability, and deterministic readiness requirements for the supported accounting pathways.	Must
FR14	Persist each new calculation as a versioned record with frozen evidence/input/result/readiness context and supersession metadata so later evidence or calculation changes do not silently rewrite historical numerical payloads.	Must
FR15	Allow eligible calculation versions to enter an internal review workflow with submissions, reviewer assignments, review events, findings, comments, and notifications.	Should
FR16	Provide portfolio and operational views for project/calculation summaries, methodology/source coverage, configured AI providers, background jobs, and worker status.	Should

ID	Requirement	Priority
FR17	Enforce organization scoping and project-role permissions for supported workflows while allowing permitted updates to editable project/field metadata without changing immutable accounting-pathway identity.	Must
FR18	Execute long-running signal and AI work through the durable database-backed queue with worker heartbeat, retry, cancellation, and failure states rather than relying on in-request synchronous execution.	Must

3.3 Non-functional Requirements

Table 2: Non-functional requirements and verification targets

ID	Category	Requirement	Verification
NFR1	Security & Identity	Authentication secrets and credentials must be protected; production configuration must reject weak/default JWT secrets, and account tokens must be validated against the current active user and token-version state.	Authentication/account-access tests plus configuration and security-code review.
NFR2	Authorization & Tenant Isolation	Tenant identity must come from authenticated server state, and project/role authorization must be enforced for protected operations; any endpoint that remains organization-scoped rather than uniformly project-member-scoped must be documented as a limitation.	Tenant/auth regression tests and endpoint-level authorization review.
NFR3	Auditability & History	Saved calculations must retain frozen evidence/input/result/readiness context and a version chain sufficient to review historical results without silently recomputing them from current evidence.	Calculation/snapshot/export regression tests and historical-version comparisons.
NFR4	Reproducibility	Deterministic accounting and scoped fingerprints must be reproducible from stored values and stable canonical serialization; software/model versions and provenance limitations must be disclosed where complete executable provenance is not yet captured.	Golden-value tests, fingerprint tests, snapshot/version tests, and code review of version/provenance fields.
NFR5	Methodology Integrity	Unsupported, incomplete, or reviewer-dependent methodology requirements must remain explicit and must not be silently interpreted as satisfied, zero, or externally verified.	Methodology/readiness tests and review of blocking/unsupported requirement behavior.
NFR6	External-Service Resilience	External-service failures (Earth Engine, email, storage, AI provider, worker availability) must degrade through actionable application/job states rather than exposing raw failures as successful results.	Job/API tests, fault-path review, and operational/manual service checks.
NFR7	Availability & Job Durability	Long-running analytics and AI work must be decoupled from request lifetimes through the durable queue, with persisted pending/running/completed/failed/cancelled state, retries where supported, and worker heartbeat visibility.	Durable-job tests, worker CLI checks, and job-state UI/API verification.
NFR8	Maintainability	Domain packages must remain separated by capability (accounts, projects, evidence, methodology, carbon, signals, jobs, AI, persistence, reporting) so API/frontend changes do not become calculation authority.	Architecture/code review plus full regression suite.
NFR9	Portability & Migration Transparency	The system must support local/test SQLite and hosted PostgreSQL configurations without changing domain behavior; deployment-specific limitations and unverified migration paths must be stated explicitly.	Fresh/repeated SQLite initialization tests, representative legacy-upgrade test, and

ID	Category	Requirement	Verification
			deployment/migration review.
NFR10	Privacy & Storage	Private evidence/model artifacts must be accessed through authenticated application paths, use sanitized storage keys/checksums, and avoid exposing tenant files as public static assets; shared-storage/backup limitations must be disclosed.	Attachment/storage tests, authenticated-download review, and backup/restore script review.
NFR11	Usability & Accessibility	Important UI states—loading, queued, running, blocked, permission denied, validation failure, completed, failed and retry—must be visible and understandable without relying on color alone.	Frontend tests, ESLint, responsive/UI review, and final-state screenshot inspection.
NFR12	Scientific Validity	Satellite/ML outputs must be represented as experimental/supporting evidence unless independently validated; software tests or model-versus-threshold metrics must not be described as proof of real-world AWD accuracy.	Review of detector-label provenance plus explicit scientific-validation status in the report.
NFR13	Deployment & Operations	Supported deployment topology, external dependencies, storage/database configuration, and backup/recovery assumptions must be documented separately from what has actually been verified live.	Deployment checks, configuration review, worker/CLI checks, and explicit verified-vs-supported reporting.
NFR14	Testability	The final verification process must pass the current backend/core regression suite and frontend test/build/type checks, while any unexecuted scientific, browser-E2E, PostgreSQL, provider, or external-service validation remains explicitly separate.	Re-run on 7 October 2026: backend 177 passed, 0 failed, 0 skipped; frontend tests, build and TypeScript pass; ESLint 0 errors, 5 warnings.

3.4 Stakeholders and Viewpoints

Table 3: Stakeholders and viewpoints

Stakeholder	Interest
Carbon Project Developer / Project Lead	Creates and manages project context, registers fields, coordinates evidence, runs scoped calculations, reviews readiness/history, and prepares exports; project contributors/viewers receive narrower role-based access.
Researcher / Data Scientist	Builds/evaluates experimental AWD and crop models, studies satellite/threshold behavior, and distinguishes model agreement from independently validated agronomic accuracy.
Internal Reviewer	Reviews eligible calculation versions inside Terra-Audit, records findings/comments/events, and checks evidence/readiness context; this role is internal and is not a substitute for a VVB.
Program Operator (farmer cooperative / NGO)	Coordinates enrollment and field/evidence collection across participating farms and uses the platform to prepare project records for downstream review.
Farmer / Landholder	Owns or manages enrolled land and provides or confirms agronomic, production, livestock, soil, irrigation, and other field evidence used by project staff.
System Operator / Administrator	Maintains application configuration, methodology sources, provider permissions, worker/queue operations, database/storage settings, and operational/backup tooling.
External Validation/Verification Body (VVB) / Verra Process	Independently evaluates methodology applicability, evidence, monitoring, calculations, safeguards and verification requirements outside the application.

Stakeholder	Interest
	Terra-Audit internal review and estimated quantities do not constitute VVB verification or Verra-issued VCUs.
External Data and AI Services	Google Earth Engine/Copernicus provide satellite observations, while configured AI/email/storage services support optional platform functions; their availability and credentials constrain runtime behavior.

3.5 Quality Function Deployment

The final QFD groups the functional requirements in Table 1 by user value without duplicating their full specification. Normal requirements are the essential access, evidence, accounting, readiness, versioning, export and durable-job capabilities (FR1, FR2, FR5-FR8, FR11, FR13, FR14, FR17, FR18). Expected requirements cover traceable signal processing, history/portfolio, internal review and operational visibility (FR3, FR4, FR12, FR15, FR16). Exciting requirements are the experimental ML and bounded generative-AI capabilities (FR9, FR10), which remain outside deterministic accounting and external verification authority.

Traceability runs from stakeholder need → QFD category → FR ID and priority → implementing component (Chapter 7) → verification evidence (Chapter 12).

3.6 Usage Scenarios and User Stories

US1 — Project lead: create/select project context, register fields, assign members, organize evidence and save versioned calculations so every result remains tied to an authorized project and field.

US2 — Rice/ALM project user: collect the pathway-specific signal or structured practice/SOC/production/leakage evidence needed for the scoped VM0051 or VM0042 calculation, while retaining source context and blocking incomplete requirements.

US3 — Reviewer: inspect methodology readiness, frozen calculation versions, evidence context, findings and history so unresolved requirements remain visible and later evidence does not silently rewrite historical numerical results.

US4 — Researcher or authorized AI user: compare experimental threshold/ML outputs or request grounded draft explanations while keeping model agreement and AI text distinct from scientific validation, accounting authority and external verification.

US5 — Administrator: monitor methodology sources, provider permissions, jobs and worker state so background/external-service failures can be diagnosed without being presented as successful work.

4. Scenario-Based Modeling

4.1 Use Case Diagrams

Use-case modeling describes the externally visible goals that Terra-Audit actors achieve. Primary actors are project users (lead, contributor, viewer), researchers, internal reviewers and administrators. Google Earth Engine, the AI provider and the email service are external systems;

the database and storage are internal and are not modeled as actors. VVB and Verra processes lie outside the system boundary.

4.1.1 Use Case Level 0: System Context

At Level 0 the whole platform is a single use case (Fig 2). Every human actor interacts with Terra-Audit, which in turn depends on three external systems.

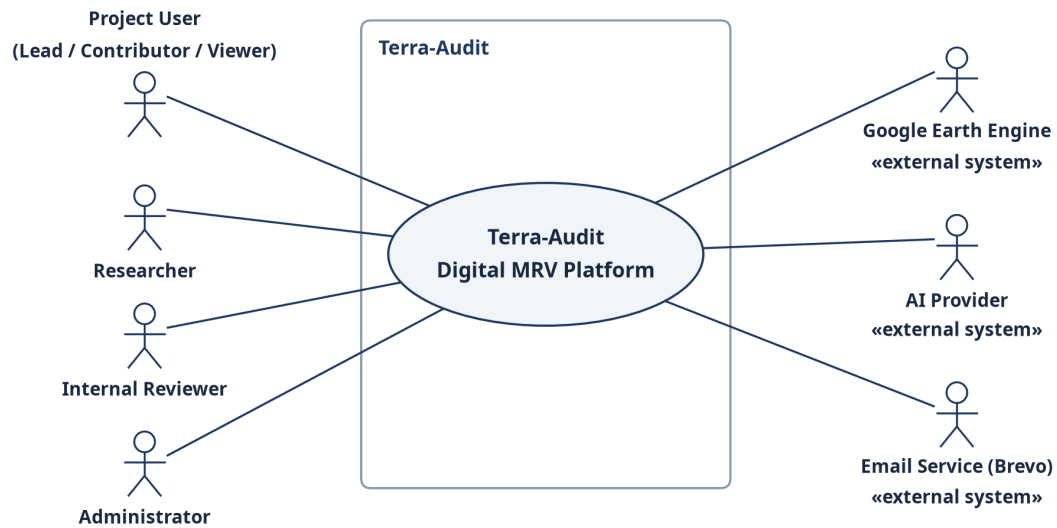


Fig 2: Use Case Level 0 — Terra-Audit as a single system with its actors and external systems.

4.1.2 Use Case Level 1: Modules of Terra-Audit

Level 1 decomposes the platform into ten user-goal modules (Fig 3); Table 4 maps each module to its actors.

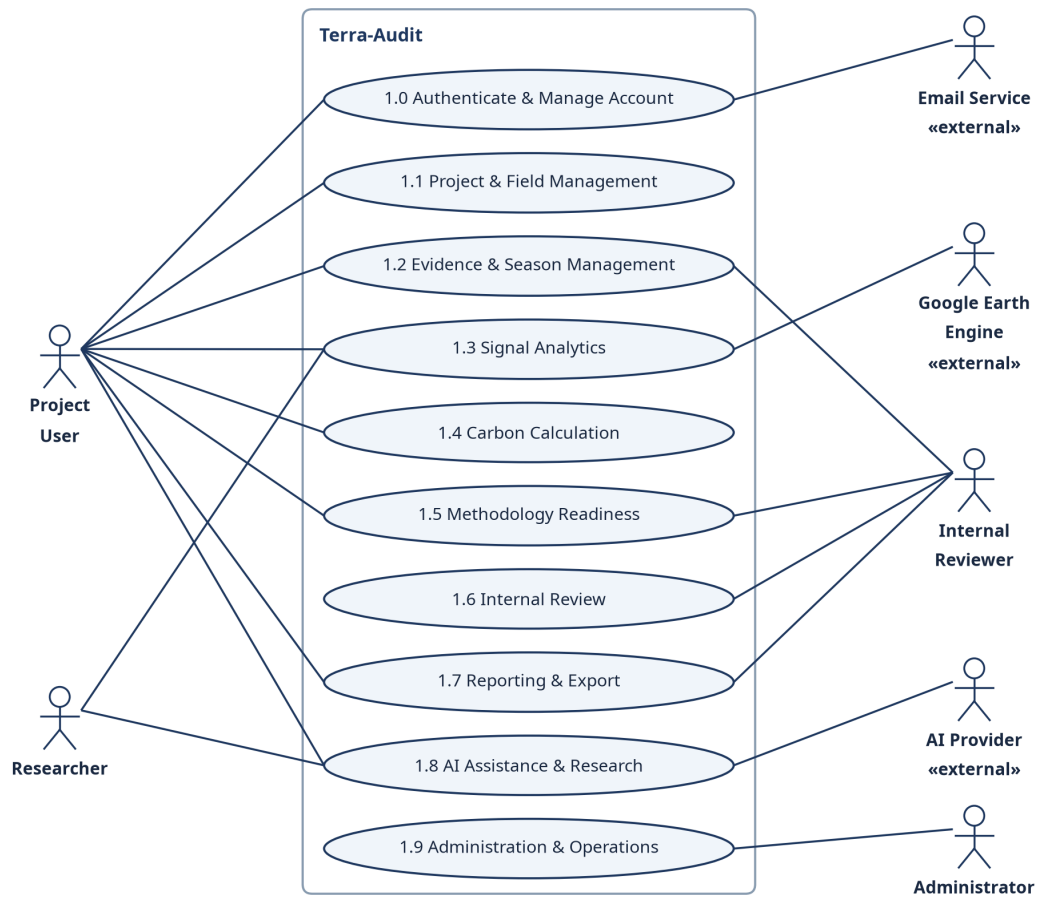


Fig 3: Use Case Level 1 — the ten user-goal modules and their actors.

Table 4: Use-case inventory and actor mapping

ID	Use Case	Primary Actor	Secondary Actor / System	Figure
L0	Terra-Audit Digital MRV Platform	All human actors	Earth Engine, AI provider, email	Fig 2
1.0	Authenticate & Manage Account	Project User, Administrator	Email service (Brevo)	Fig 3
1.1	Project & Field Management	Project Lead / Contributor	—	Fig 3
1.2	Evidence & Season Management	Project User, Internal Reviewer	—	Fig 3
1.3	Signal Analytics	Project User, Researcher	Google Earth Engine	Fig 3
1.4	Carbon Calculation	Project User	—	Fig 3, Fig 4
1.5	Methodology Readiness	Project User, Internal Reviewer	—	Fig 3
1.6	Internal Review	Internal Reviewer, Project Lead	—	Fig 3
1.7	Reporting & Export	Project User, Internal Reviewer	—	Fig 3
1.8	AI Assistance & Research	Project User, Researcher	AI provider	Fig 3
1.9	Administration & Operations	Administrator	Durable job queue / worker	Fig 3

4.1.3 Use Case Level 2: Carbon Calculation

Fig 4 refines module 1.4. Preview and commit both include input selection; a commit additionally builds a frozen evidence snapshot, runs the pathway engine (VM0051 for rice_awd, VM0042 with VMD0054 leakage for cropland ALM) and evaluates readiness. Reusing saved signal evidence and superseding a prior version are optional extensions.

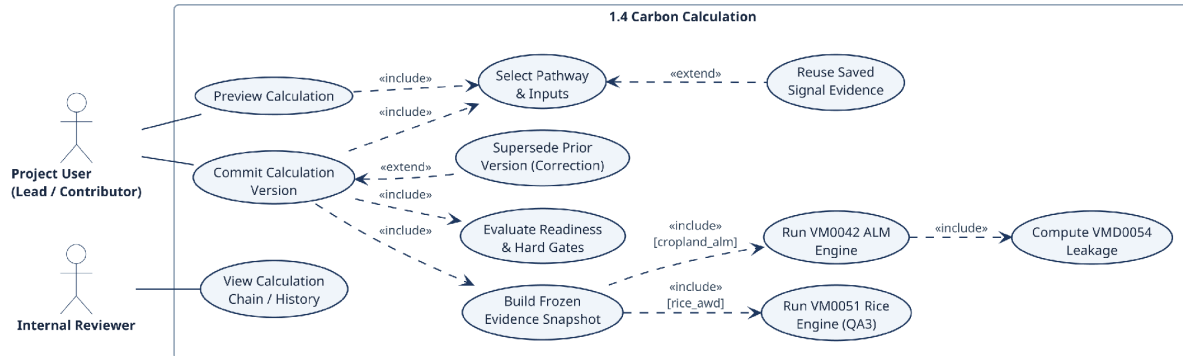


Fig 4: Use Case Level 2 — Carbon Calculation (1.4) with «include» and «extend» relationships.

4.1.4 Use Case Descriptions

Table 5 specifies the actor, precondition, normal flow, alternative/error flow and postcondition of each Level-1 use case.

Table 5: Detailed use-case specifications

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
Authenticate & Manage Account	Any user; Administrator	Registered or invited email address	Request OTP or sign in → bcrypt check → JWT stored in httpOnly session → organization and role loaded	Wrong credentials → 401; revoked token version → sign in again; email not configured → OTP unavailable	Authenticated session bound to the user's organization
Project & Field Management	Project Lead / Contributor	Authenticated organization/project context.	Create/select project → manage members → register field/geometry → assign project/farm membership → update permitted metadata.	Invalid geometry, duplicate/invalid input, or insufficient permission → no write and actionable error.	Authorized project/field context persisted.
Evidence & Season Management	Project User / Internal Reviewer	Field exists; authorized evidence context.	Create/correct season → observations/practice events → attachments → soil/lab/custody → production/livestock evidence.	Validation, permission, storage, or missing-required-data errors remain visible; corrections append history rather than silently replacing prior versions.	Versioned evidence state available to readiness/calculation workflows.
Signal Analytics	Project User / Researcher	rice_awd field and monitoring window selected.	Reuse compatible active/cache result where possible → otherwise enqueue	Earth Engine, worker, empty-data, detector, or	Completed signal result or explicit failed/cancelled

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
			signal_run → worker queries Earth Engine → process threshold/ML detector → store result.	authorization failure → job/error state; no successful result is fabricated.	d state is available.
Carbon Calculation	Project User	Registered field plus pathway-specific inputs/evidence.	Select pathway and inputs → optionally use saved signal → build snapshot → run VM0051 or VM0042/leakage → evaluate gates/readiness → save version.	Invalid inputs, unsupported/applicability conditions, leakage/SOC hard blocks, or unresolved readiness prevent a ready-for-review state.	Versioned draft or ready-for-review calculation is persisted.
Methodology Readiness	Project User / Internal Reviewer	Active methodology bundle and project/field context.	Resolve bundle/applicability → evaluate registered requirements/corrections → calculate fingerprint/readiness → expose blockers and reviewable items.	Unsupported or automated-only requirements cannot be silently overridden; stale evidence changes the fingerprint/state.	Current readiness and blocking evidence context is displayed/stored.
Internal Review	Project Lead / Internal Reviewer	Specific calculation version is ready_for_review.	Submit calculation → assign reviewer → record review events/findings/comments → resolve or track findings.	Blocked/draft calculation cannot enter normal review; authorization failures are rejected.	Version-pinned internal review trail is retained.
Reporting & Export	Project User / Internal Reviewer	Calculation/history context exists and user is authorized.	Select the relevant verification or calculation record → choose an export format actually available for that record → inspect history/portfolio context → download.	Unsupported export, missing context, or legacy provenance limitation is disclosed rather than silently substituted.	Supported evidence/report artifact is downloaded with stated provenance limits.
AI Assistance & Research	Researcher / Authorized User	Authorized project context plus evidence/methodology context or a research dataset.	Build/evaluate AWD or crop models, or request AI explanation → build deterministic packet → provider/job → validate citations/numbers → retain draft.	Missing provider/permission/data, oversized packet, or failed validation → warning/failed job; no authoritative decision is written.	Research metrics or a validated human-review draft is available.
Administration & Operations	Administrator	Administrative authorization.	Manage methodology sources/provider permissions → inspect jobs/workers → retry/cancel supported jobs → use	Permission, stale worker, external-service, or configuration failure remains visible and does	Operational configuration and monitoring state are updated or an

Use Case	Actor(s)	Precondition / Trigger	Normal Flow	Alt. / Error Flow	Postcondition
			operational/backup tooling.	not masquerade as success.	explicit failure remains.

4.2 Activity Diagrams

The activity diagrams describe the implemented control flow. Swimlanes separate the user, the server and the durable worker; blocked states are shown explicitly so that failed services or incomplete methodology requirements never appear as successful MRV outcomes.

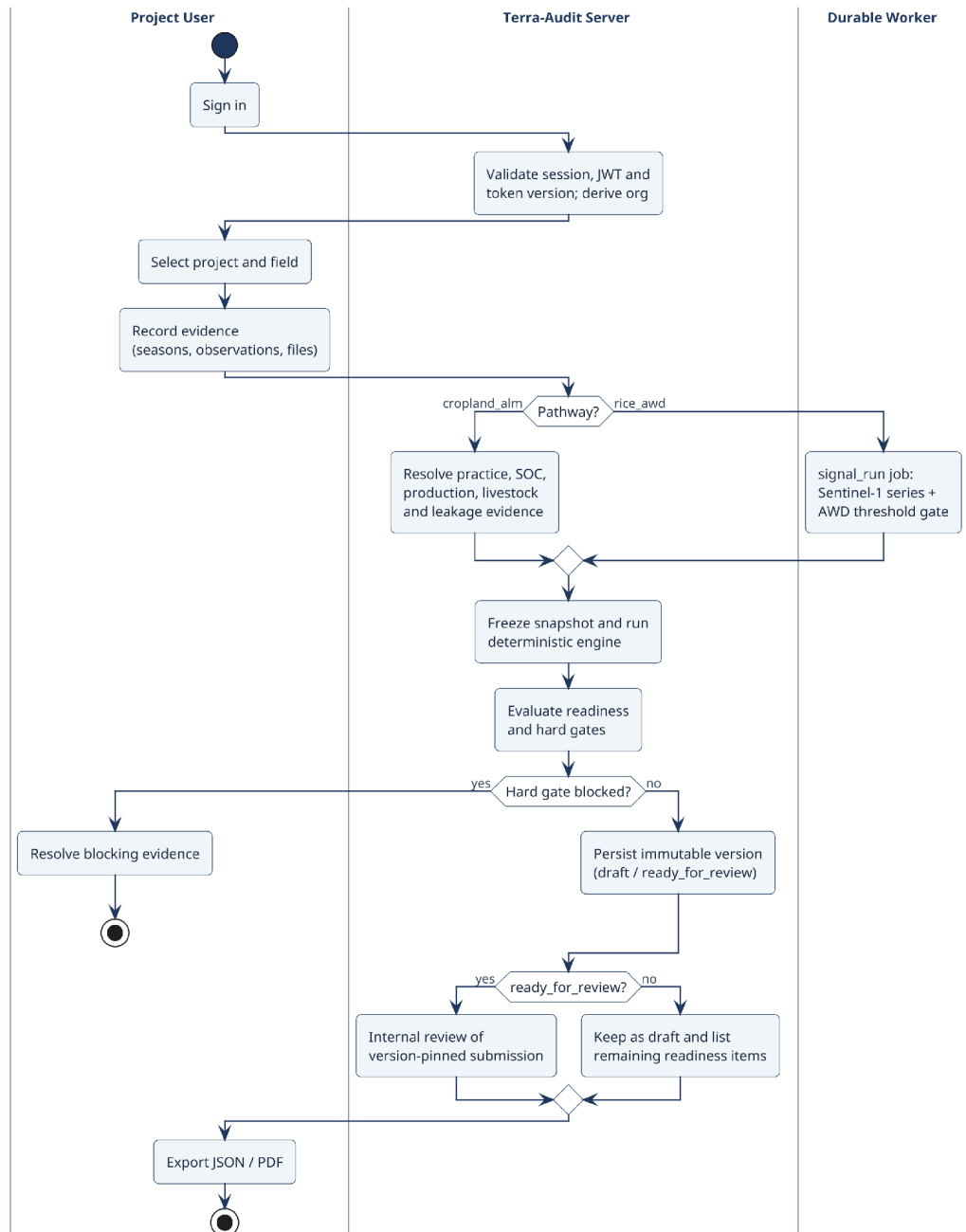


Fig 5: Overall activity diagram with swimlanes for the project user, the Terra-Audit server and the durable worker.

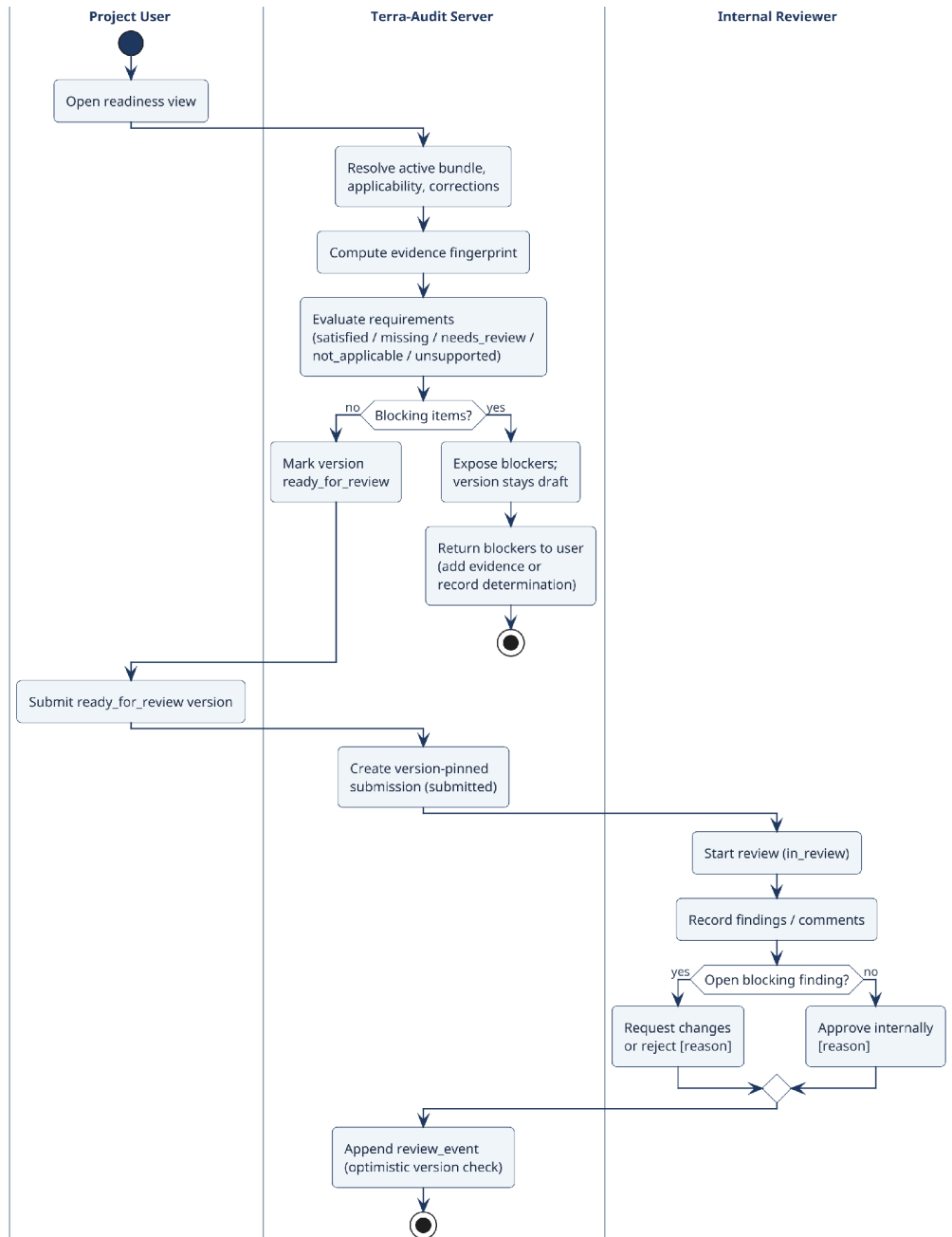


Fig 6: Activity diagram — methodology readiness and internal review with swimlanes.

4.2.1 Cropland ALM Detailed Activity Flow

The final ALM flow begins with an organization/project-scoped cropland_alm_vm0042 field and dated project membership. Users record baseline/project crop seasons and practice schedules, optional supported livestock schedules, production records, soil sampling plans/strata/samples, laboratory results, custody events, and reviewed SOC mappings (or explicitly identified legacy aggregate SOC evidence). A project/bundle/reporting-period leakage assessment is then saved. The calculation workflow builds a frozen snapshot containing the relevant practice, production, leakage and SOC evidence; the restricted VMD0054 v1.1 adapter computes the supported displacement-leakage term; and the scoped VM0042 v2.2 engine evaluates SOC/non-SOC terms, uncertainty, allocation and buffer logic. Readiness and engine hard gates are evaluated before a versioned calculation is persisted. Only a ready_for_review version can enter internal review.

Snapshot-based ALM PDF and calculation JSON exports can then be generated. Current restrictions—including single-field leakage scope, unsupported mitigation Step 2, nonannual SOC blocking where unresolved annualization applies, and incomplete sequential-period cumulative-state reconciliation—must remain visible rather than being represented as complete methodology coverage.

5. Data-Based Modeling

5.1 Entity-Relationship Diagram

Terra-Audit uses a domain-oriented relational schema. Fig 7 shows the core entities in crow's-foot notation with their primary keys and principal reference attributes; Fig 8 details the calculation and internal-review tables. Most tables are tenant-scoped through a composite (org_id, id) primary key. Relationships are logical: the SQLite schema does not declare FOREIGN KEY constraints, so scoped identifiers, application queries and authorization checks enforce referential integrity.

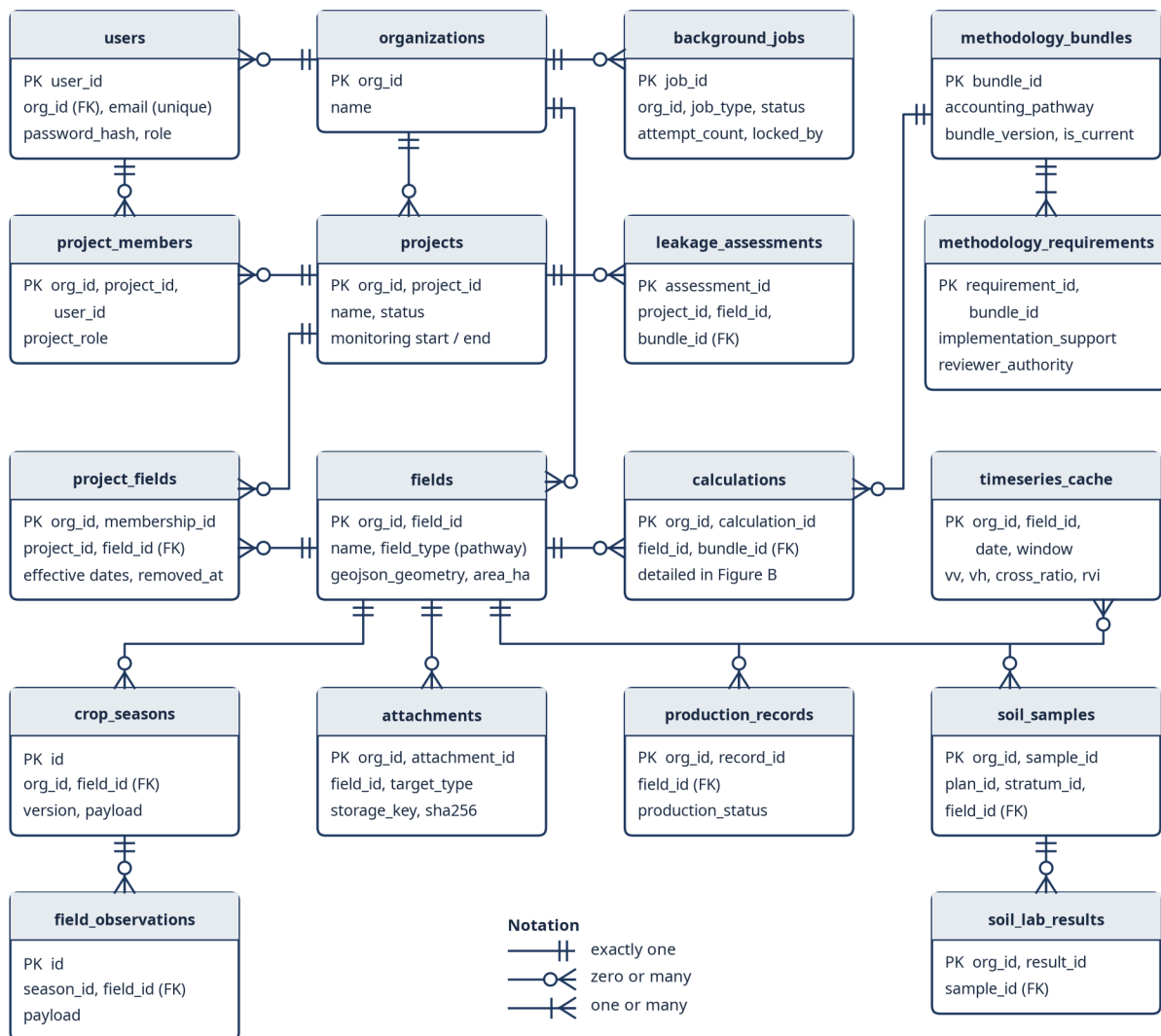


Fig 7: Entity-relationship diagram (crow's-foot notation) of the core persistent entities.

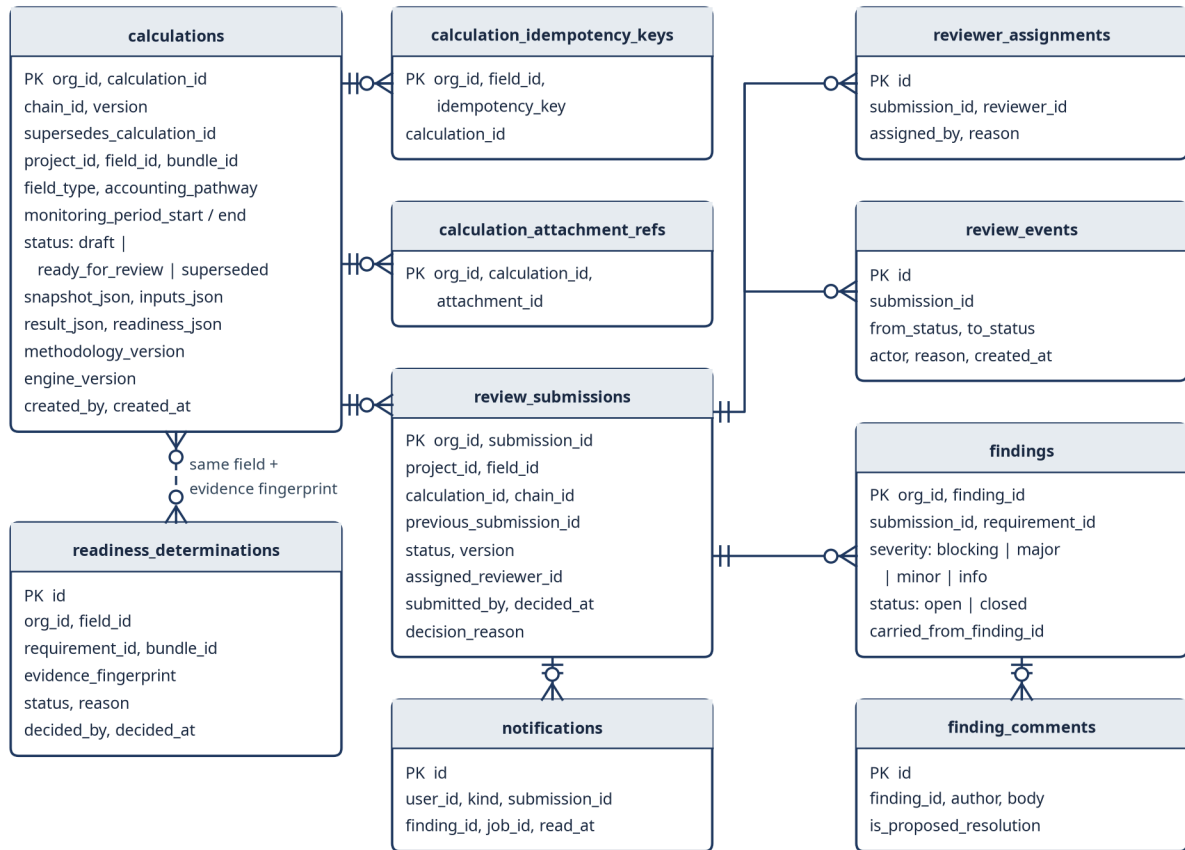


Fig 8: Entity-relationship diagram of the calculation, readiness and internal-review tables.

5.1.1 Relationship Summary

Identity and project context — User-to-project access is represented through ProjectMember with explicit roles. Project-to-field membership is represented through effective-dated ProjectField records, so a field is not reduced to a single project_id and assignment history can be retained.

Evidence and sampling — CropSeason, FieldObservation, ObservationReview and PracticeEvent use append-oriented field/season records. Attachments remain field-scoped and reference supported targets. The ALM soil path links sampling plans, optional strata, samples, laboratory results, custody events and reviewed SOC mappings; production records and leakage assessments provide additional field/project evidence.

Calculation provenance — Calculation records are versioned and immutable at the application level: chain_id and supersedes_calculation_id preserve correction history, while snapshot_json, inputs_json, result_json and readiness_json freeze the committed context. CalculationAttachmentRef retains protected attachment links; legacy credit history remains distinguishable from snapshot-backed calculations.

Methodology reference data — MethodologyDocument, MethodologyBundle, bundle-document membership and MethodologyRequirement form the shared registry. Tenant-specific ProjectMethodologyApplicability records bind a project/pathway to a bundle and effective period; correction/confirmation records remain separate from global source data.

Internal review — ReviewSubmission pins one immutable calculation version. Reviewer assignments, lifecycle ReviewEvents, Findings and finding comments/resolution activity belong to the internal workflow and never represent external VVB verification, Verra registration or registry issuance.

5.2 Database Schema

Table 6: Persistent database schema by domain

Domain	Important Entities / Tables	Purpose
Accounts & Identity	organizations, users, pending_registrations, account_links, account_token_versions, account_rate_limits	Tenant identity, account access/recovery, token revocation/versioning, and account-level throttling.
Projects & Land	projects, project_members, project_membership_events, farms, fields, project_fields, farm_fields, quantification_units	Project roles and dated land/field memberships; field records carry the registered accounting pathway and geometry context.
Monitoring & Evidence	crop_seasons, field_observations, observation_reviews, practice_events, monitoring_runs, data_quality_issues, attachments	Versioned crop-season history and structured monitoring/evidence records, including private evidence-file metadata.
Soil / Production / ALM	soil_sampling_plans, soil_strata, soil_samples, soil_lab_results, soil_custody_events, soc_evidence_reviews, alm_practice_schedule, alm_livestock_schedule, production_records, leakage_assessments	Structured ALM evidence for reviewed SOC, practices, production, supported livestock inputs, and restricted leakage assessment.
Calculations & Review	calculations, calculation_idempotency_keys, calculation_attachment_refs,	Versioned calculation snapshots/results/readiness plus version-pinned internal review and findings history.

Domain	Important Entities / Tables	Purpose
	readiness_determinations, review_submissions, reviewer_assignments, review_events, findings, finding_comments	
Methodology	methodology_documents, methodology_bundles, bundle-document mappings, methodology_requirements, project_methodology_applicability, methodology_chunks/FTS, correction links/confirmations	Methodology/version/correction registry, retrieval index, applicability context, and requirement definitions used by readiness and AI context.
Jobs / AI / Satellite	background_jobs, job_batches, workers, ai_records, ai_deployments, ai_provider_permissions, timeseries_cache, timeseries_cache_versions; lazy ai_dataset_rows / ai_dataset_versions	Durable asynchronous work, worker state, satellite cache/versioning, provider/model records, and lazily created research-dataset tables.

The SQLAlchemy connection layer supports SQLite and PostgreSQL. SQLite startup initialization contains historical additive ALTER TABLE operations and selected rebuild/copy migrations; the PostgreSQL path uses final-shape DDL plus module-specific additive changes. There is no Alembic-style versioned migration ledger. In the final software checks, fresh SQLite initialization, repeated initialization/restart, a representative legacy SQLite upgrade, and SQLite integrity checking passed. Remote PostgreSQL migration/runtime verification was not completed, and the existing SQLite-to-PostgreSQL transfer script covers only a legacy subset of the current schema.

5.3 Dataset Description (AI-Assisted Detection)

The classical AWD dataset is built from cached rice time-series and labels generated by the rule-based threshold detector rather than by independent field observations. Feature engineering uses satellite/time-series values while excluding the gate's own decision flags as direct predictor features. Random Forest and XGBoost therefore measure model-versus-threshold agreement; their metrics cannot be interpreted as accuracy against independently observed irrigation events.

The implemented evaluation harness now requires field identifiers and uses leakage-safe GroupKFold splits so the same field is not placed in both training and validation folds. Pooled out-of-fold precision, recall, F1, confusion-matrix and ROC-style outputs are still explicitly labeled as threshold-gate agreement. Persisted model bundles also record the satellite processing version and field-grouped split strategy so stale feature versions are rejected during loading.

The principal validation risk is no longer observation-level fold leakage; the current implementation uses field-grouped evaluation. The larger limitation is label provenance: the target labels still come from the same threshold detector that the models are intended to imitate. Sparse drydown events, Sentinel revisit gaps, phenology effects, and the absence of independently observed water-level/irrigation events limit how strongly the resulting metrics can be interpreted.

Future scientific validation should compare detected flood/drydown events against independent ground truth such as water-level-tube observations, irrigation logs, or partner-verified field observations and report event-level precision/recall, timing error, false drydown rate, and

performance across seasons/locations. Managed crop models are a separate exploratory feature using reviewed crop-identity labels; neither that work nor the AWD classifiers establish methodology-authoritative AWD compliance.

6. Class-Based Modeling

6.1 Analysis Classes

Candidate classes were identified with the grammatical-parse technique [17]: nouns in the usage scenarios (Section 3.6) and the SRS were listed and then filtered by whether they retain information, provide needed services, have multiple attributes and share common operations. Table 7 summarizes the decisions.

Table 7: Analysis-class identification from scenario nouns

Candidate noun(s)	Decision	Resulting design element
User, organization, project member	Accepted	accounts / projects modules; users, project_members
Project, farm, field	Accepted	projects repository; fields, project_fields
Crop season, observation, practice event	Accepted	evidence modules (append-only records)
Soil sample, lab result, production record	Accepted	soil and production evidence modules
Satellite image, backscatter time series	Accepted	SpatialDataEngine; timeseries_cache
Flooding / drydown event, phenology	Accepted	AdaptiveAWDGate
Emission reduction, carbon credit, leakage	Accepted	CarbonAssetEngine, AlmCarbonEngine, VMD0054
Calculation, snapshot, version	Accepted	snapshot / calculations module
Methodology, bundle, requirement	Accepted	methodology registry and readiness
Review, finding, comment	Accepted	reviews module
Job, worker; explanation, provider	Accepted	jobs queue + worker; AI packets / validator
Date, area, location, district	Rejected (attribute)	attributes of field / season
Dashboard, page, button	Rejected (interface)	frontend components only
VVB, Verra registry	Rejected (external)	outside the system boundary

6.2 Class Cards (CRC Model)

CRC cards record each design class's responsibilities and collaborators (Fig 9). Only implementation-backed classes and modules are shown; method-level operations appear in the design class diagram (Fig 10).

SpatialDataEngine Responsibilities Query Sentinel-1 GRD (IW, VV+VH) Derive cross-ratio and RVI bands Median-reduce, Savitzky-Golay smooth Collaborators Google Earth Engine, timeseries_cache	AdaptiveAWDGate Responsibilities Z-score flood detection ($z < -0.8$) Detect drydown events (1.2 sigma) Extract sowing / harvest dates Collaborators SpatialDataEngine, ML predictor	CarbonAssetEngine Responsibilities VM0051 QA3 CH4 scaling factors N2O irrigation penalty, uncertainty Block results above 60,000 tCO ₂ e/yr Collaborators AdaptiveAWDGate, snapshot
AlmCarbonEngine Responsibilities VM0042 SOC stock change, ER / CR Non-SOC, livestock, buffer terms Block missing leakage evidence Collaborators VMD0054 leakage, AlmPracticeValidator	Snapshot & Calculations Responsibilities Freeze evidence / inputs / results Idempotent, versioned commit Supersede prior version on correction Collaborators Carbon engines, readiness	Readiness Responsibilities Resolve bundle and applicability Evaluate requirement statuses Compute evidence fingerprint Collaborators Methodology registry, corrections
Reviews Responsibilities Version-pinned submissions Guarded state transitions Findings, comments, notifications Collaborators calculations, users	Job Queue & Worker Responsibilities Persist and claim jobs atomically Heartbeat, lease, retry, cancel Dispatch typed job handlers Collaborators Signal / AI / ingest handlers	AI Packet & Validator Responsibilities Build bundle-scoped context packet Run 14 server-side output checks Store draft for human review Collaborators AI providers, methodology library

Fig 9: CRC cards for the nine core design classes.

6.3 Design Class Diagram

The implementation is modular but is not a uniform class/service/repository hierarchy. Design responsibilities are distributed across domain packages and a smaller number of classes/helpers. Important software-design units include account/authentication primitives in `src/accounts` and `backend/security.py`; project and internal-review repositories in `src/projects`; evidence modules for seasons, monitoring, soil and production; methodology registry/library/readiness modules; deterministic rice and ALM carbon engines; the leakage adapter; snapshot/calculation-version persistence; signal acquisition/processing; the durable job queue and worker handlers; AI packets/providers/validators and ML models; reporting; and persistence/storage.

The main authority boundary is deliberate: UI, jobs, satellite models and language models may provide inputs, analytics or explanations, but accounting values, readiness states and validation rules remain server-owned deterministic behavior. This separation reduces the risk that a probabilistic component silently changes methodology logic.

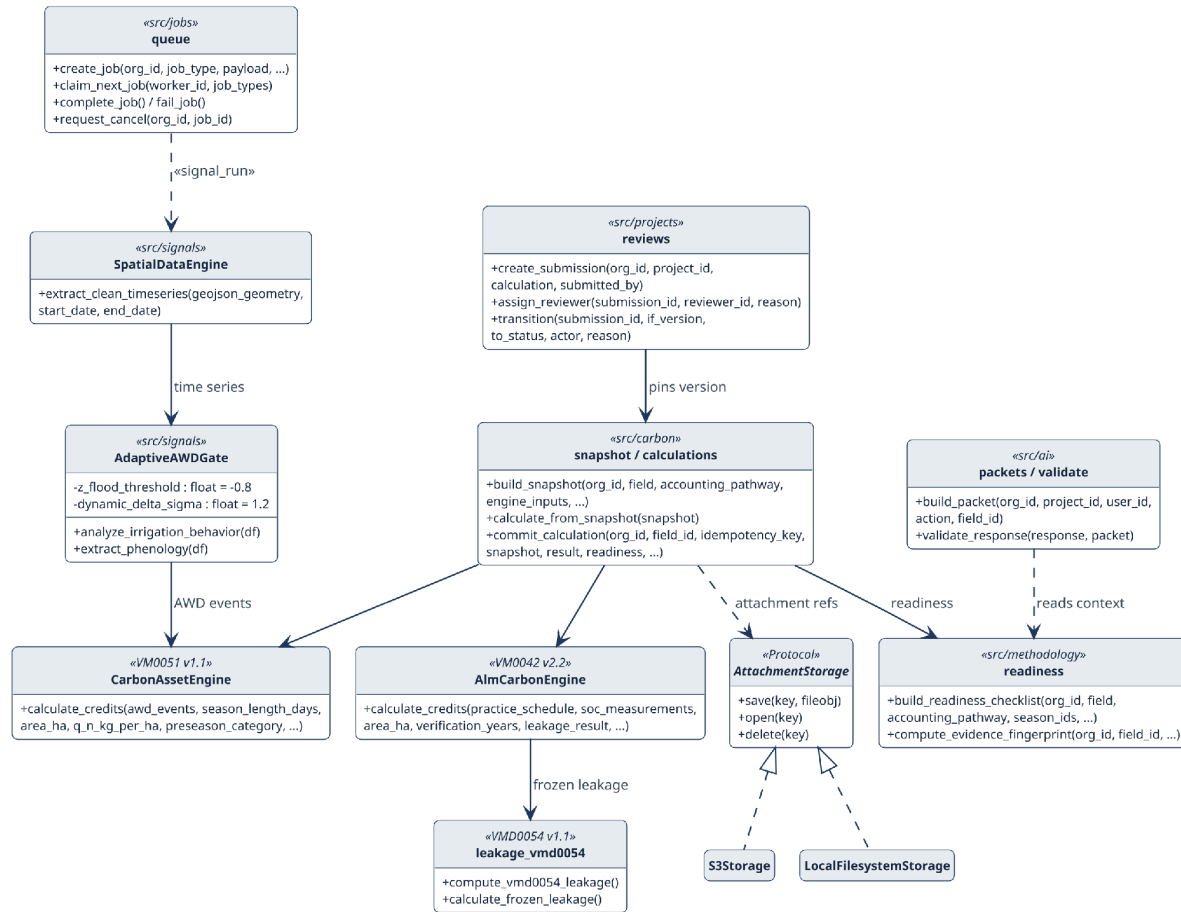


Fig 10: Design class diagram: principal classes and modules, key operations and dependencies.

6.3.1 Design Class to Implementation Mapping

The implementation is capability-oriented rather than a uniform object hierarchy, so Table 8 names the real classes and module-level operations that own each design responsibility.

Table 8: Design classes and their implementing modules

Design class	Module	Key operations	Responsibility
CarbonAssetEngine	src/carbon/rice.py	calculate_credits()	Scoped VM0051 v1.1 QA3 rice result, intermediates, hard gates
AlmCarbonEngine	src/carbon/alm.py	calculate_credits()	Scoped VM0042 v2.2 SOC/non-SOC terms, uncertainty, buffer, blocking flags
SpatialDataEngine	src/signals/earth_engine.py	extract_clean_timeseries()	Sentinel-1 VV/VH, cross-ratio and RVI series
AdaptiveAWDGate	src/signals/threshold_gate.py	analyze_irrigation_behavior(), extract_phenology()	Flood/drydown and sowing/harvest indicators
Snapshot & versioning	src/carbon/snapshot.py, calculations.py	build_snapshot(), commit_calculation()	Frozen context; idempotent immutable versions
Readiness	src/methodology/readiness.py	build_readiness_checklist()	Requirement statuses and bundle context
Internal review	src/projects/reviews.py	create_submission(), transition()	Version-pinned, guarded review transitions

Design class	Module	Key operations	Responsibility
Durable jobs	src/jobs/queue.py	create_job(), claim_next_job(), fail_job()	Persisted job lifecycle, lease, retry, cancel
AttachmentStorage	src/persistence/storage.py	save(), open(), delete()	Private local or S3-compatible artifacts
AI packet & validator	src/ai/packets.py, validate.py	build_packet(), validate_response()	Bounded context; server-side output checks
Reporting	src/reporting/reports.py	report generators	Legacy and snapshot-backed exports

The browser request boundary is separate: frontend/lib/api.ts sends actions through the Next.js catch-all API proxy, which reads the httpOnly session server-side and forwards an authenticated request to FastAPI. This keeps browser UI state outside the accounting and authorization authority boundary.

7. Architectural Design

7.1 Architectural Context Diagram

The architectural context diagram (ACD) [17] places Terra-Audit among the systems it interacts with (Fig 11). Superordinate systems use Terra-Audit's output: VVB validation/verification and the Verra registry consume exported evidence packages but are outside the system. Subordinate systems are used by Terra-Audit to function: Google Earth Engine, AI providers, the Brevo email API, the relational database and object storage. Peers exchange information at the same level: farm production spreadsheets imported as CSV and Verra methodology documents ingested into the methodology library. Actors are the human users.

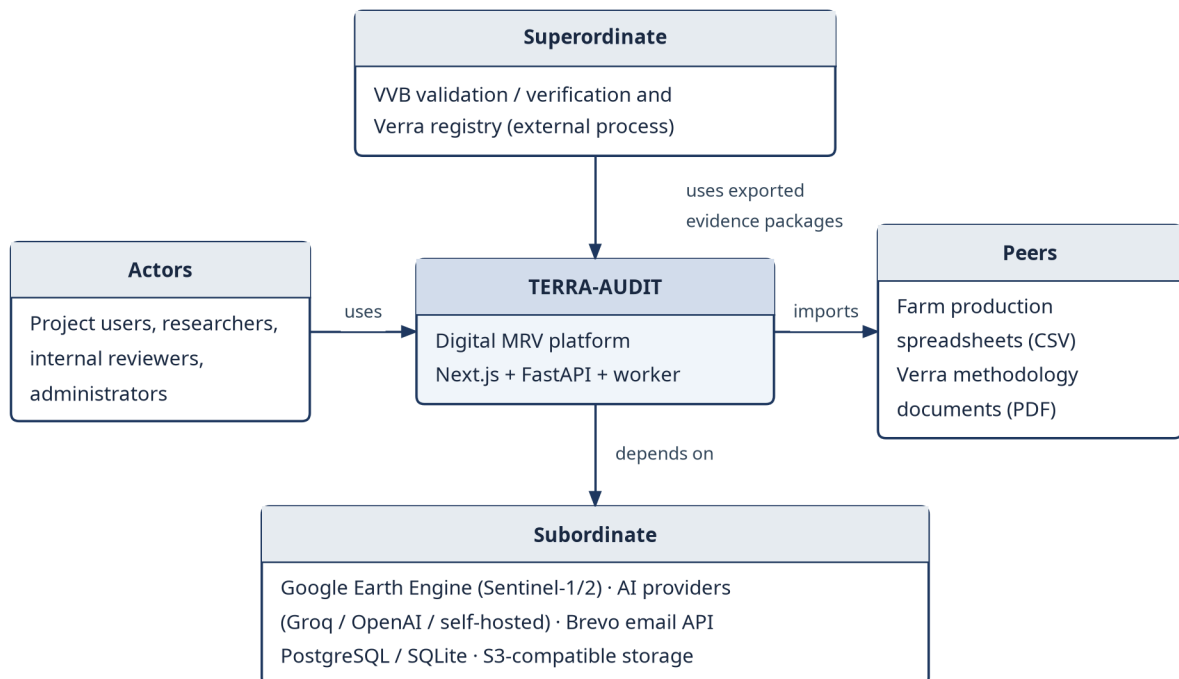


Fig 11: Architectural context diagram (ACD) of Terra-Audit.

7.2 Defining Archetypes

Two archetypes structure the system. The application as a whole follows a layered architecture (Fig 12): presentation, application/API, domain and infrastructure layers, where each layer calls only the layer below and accounting authority stays in the domain layer. The rice signal path follows the pipe-and-filter archetype (Fig 13): each filter transforms the Sentinel-1 series and passes it on, so each stage is testable on its own and a cache hit skips acquisition entirely.

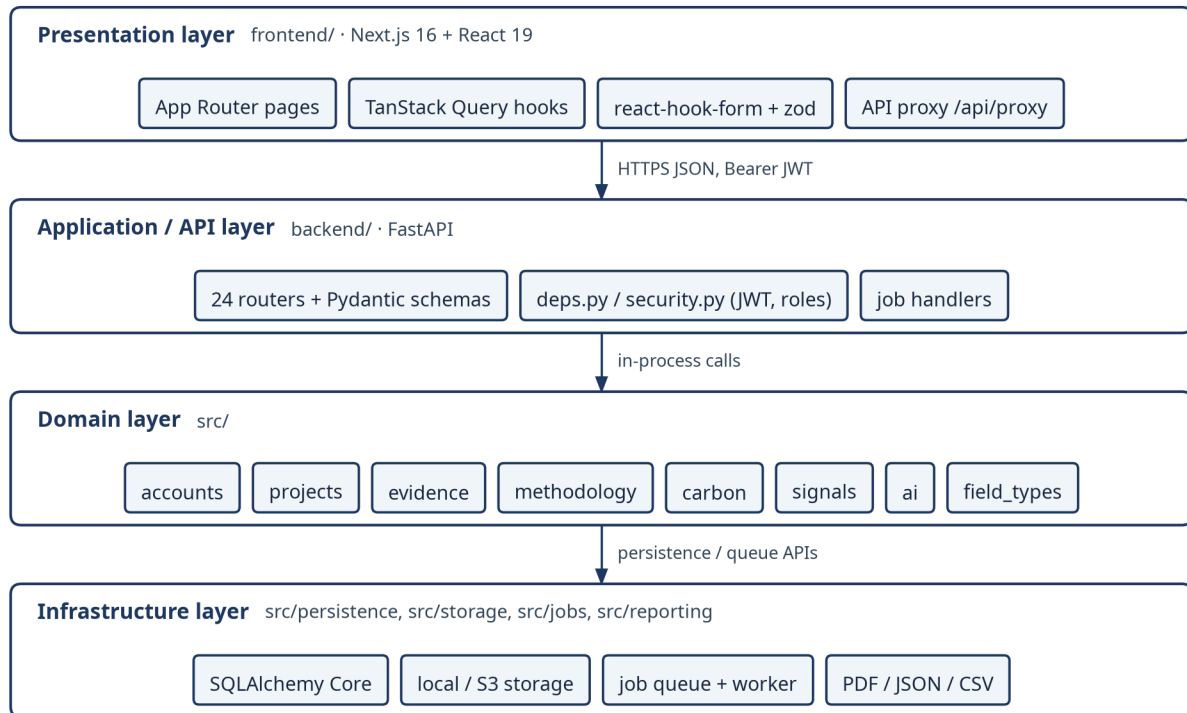


Fig 12: Layered architecture archetype instantiated for Terra-Audit.

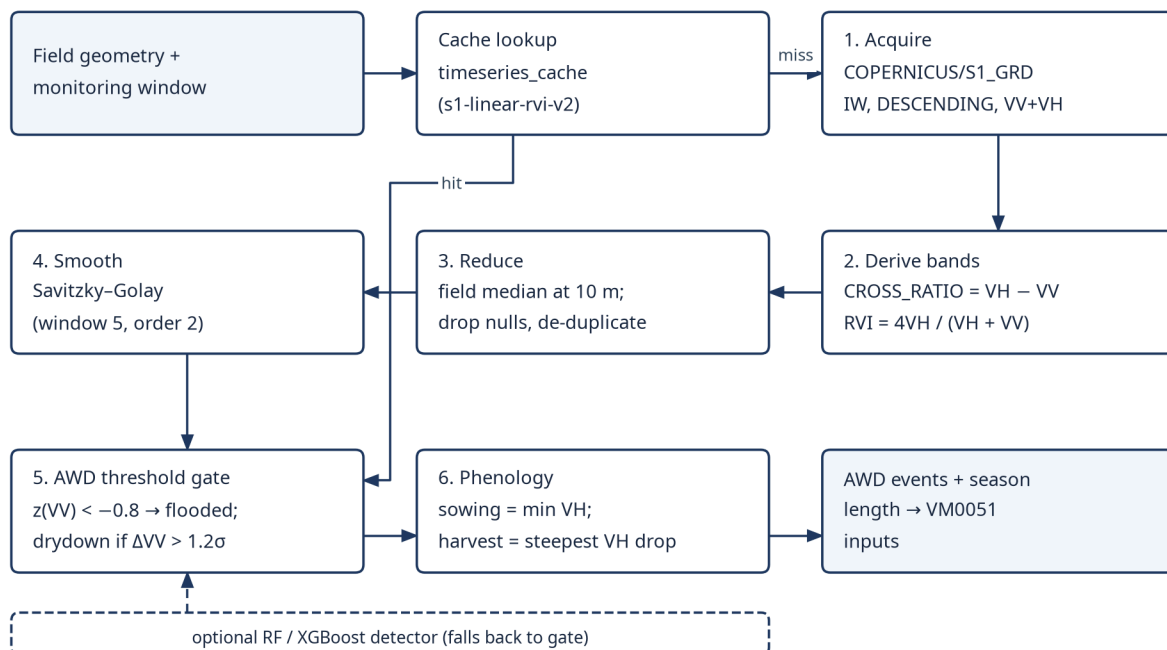


Fig 13: Pipe-and-filter archetype: the Sentinel-1 signal-processing pipeline of the rice AWD pathway.

7.3 Top-Level Components

Refining the archetypes gives the top-level components in Fig 14. A Next.js frontend calls FastAPI through an authenticated server-side proxy; FastAPI routers orchestrate shared Python domain components; long-running signal and AI work is persisted in a database-backed queue and executed by a separate worker. Deterministic accounting, readiness and server validation remain authoritative; ML and generative AI are assistive. Table 9 lists each component's responsibility.

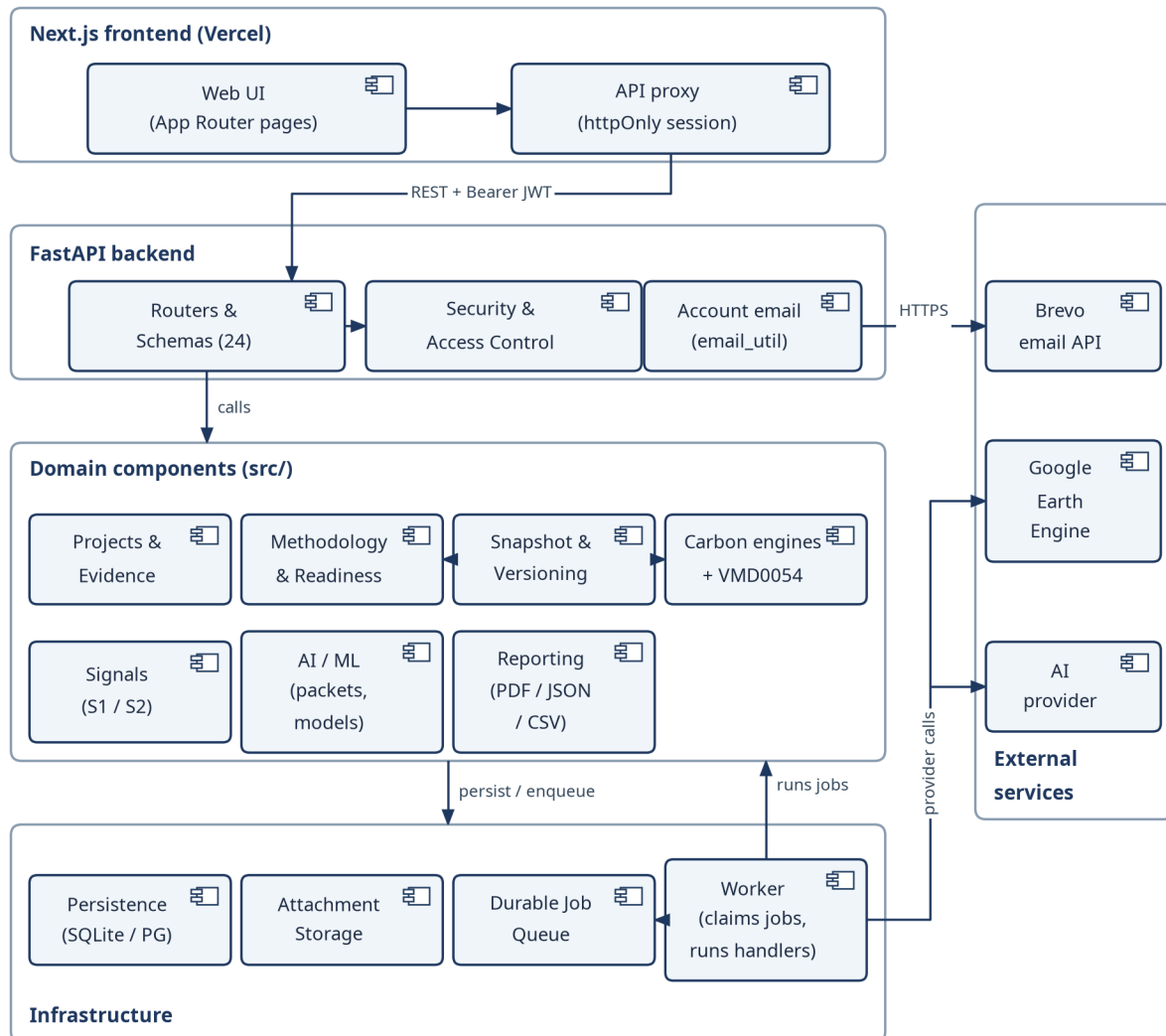


Fig 14: Top-level component diagram of Terra-Audit.

Table 9: Component responsibilities

Component	Responsibility
Next.js Frontend & API Proxy	Renders authenticated pages/components, manages client state/queries, and forwards browser requests through the server-side authenticated API proxy rather than exposing bearer tokens directly to client code.
FastAPI Routers & Schemas	Own HTTP orchestration, request/response validation, route composition, and handoff to shared domain packages.
Accounts / Security / Authorization	Implements identity, password/account-recovery flows, JWT validation, token-version checks, organization scoping, and project/admin access gates.

Component	Responsibility
Projects & Evidence	Manages projects, farms, fields, memberships, crop seasons/corrections, observations, attachments, soil/SOC, production, and internal-review domain records.
Methodology & Readiness	Maintains methodology documents/bundles/corrections, project applicability, requirement definitions, retrieval chunks, evidence fingerprints, and deterministic readiness states.
Carbon Accounting & Leakage	Runs scoped VM0051 v1.1 rice and VM0042 v2.2 ALM calculations plus the restricted VMD0054 v1.1 leakage adapter; AI does not own accounting values.
Snapshots & Calculation Versioning	Builds frozen calculation context, stores inputs/results/readiness, manages idempotency and supersession/version chains, and preserves historical numerical payloads under normal application updates.
Signals / Earth Engine	Queries and processes Sentinel-1 observations, maintains compatible cache versions, applies threshold/phenology logic, and supports selected experimental ML detectors.
Durable Jobs & Worker	Persists long-running work in the database-backed queue; separate worker processes execute signal, AI, methodology-ingestion and related jobs with heartbeat/retry/cancellation state.
AI / ML / Providers / Validation	Supports classical AWD ML, managed crop models, deterministic AI context packets, provider adapters, structured-response validation, and human-review draft explanations.
Reporting & Export	Generates legacy verification PDF/JSON/CSV artifacts, versioned calculation JSON, snapshot-backed ALM calculation PDF, and history/portfolio outputs, with different provenance guarantees for snapshot-backed and legacy paths.
Persistence & Storage	Provides SQLAlchemy-backed SQLite/PostgreSQL access plus local/private S3-compatible artifact storage; application logic enforces relationships because the verified SQLite schema declares no foreign keys.

7.4 Mapping Requirements to Software Architecture

Table 10: Mapping functional requirements to architectural elements

Requirement(s)	Architectural element	Layer
FR1, FR17	Accounts; Security & Access Control (backend/security.py, deps.py)	Application / Domain
FR2, FR5, FR6	Projects & Evidence (src/projects, src/evidence)	Domain
FR3, FR4	Signals pipeline + Durable Job Queue + Worker	Domain / Infrastructure
FR7, FR8	Carbon Engines + VMD0054 leakage adapter	Domain
FR9, FR10	AI / ML (src/ai) executed through ai_explain / ai_train jobs	Domain / Infrastructure
FR11, FR12	Reporting; calculation history and portfolio views	Domain / Presentation
FR13	Methodology & Readiness	Domain
FR14	Snapshot & Versioning	Domain
FR15	Internal review (src/projects/reviews.py)	Domain
FR16	Operations views (queue status, product readiness)	Presentation / Application
FR18	Durable Job Queue + Worker	Infrastructure

7.5 Technology Stack

Table 11: Technology stack

Layer	Technology	Purpose
Frontend	Next.js 16.3, React 19.2, TypeScript 5	App Router pages and server-side API proxy
Frontend	TanStack Query 5, react-hook-form 7 + zod 4, Tailwind CSS 4	Server state, validated forms, styling
Frontend	Leaflet 1.9 / react-leaflet 5, Recharts 3, KaTeX	Field maps and drawing, charts, equations
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic	REST API and request validation
Backend	SQLAlchemy 2 (Core), SQLite / PostgreSQL (psycopg2)	Persistence
Backend	PyJWT (HS256), bcrypt	Authentication and password hashing
Analytics	earthengine-api, pandas, NumPy, SciPy, Shapely	Sentinel-1/2 acquisition, smoothing, geometry
ML / AI	scikit-learn, XGBoost, joblib; httpx to Groq / OpenAI / self-hosted LLMs	AWD and crop models; explanations
Reporting / storage	fpdf2, pypdf, boto3	PDF export, methodology ingest, S3 storage
DevOps	Docker, docker-compose, GitHub Actions, pytest, node --test, ESLint	Build, CI and testing

7.6 Design Evolution and Alternatives

The final architecture reflects several deliberate refactorings from the midterm prototype. The Streamlit monolith was replaced by Next.js plus FastAPI so authentication, multi-user authorization, API contracts and responsive interface states could be separated from domain logic. Long-running signal and AI work moved from request-coupled execution to a durable database-backed queue with a separate worker. The old scalar/current-state calculation model evolved into versioned snapshots and readiness-aware calculation records to improve historical auditability. Methodology rules and carbon equations remain deterministic rather than being delegated to a language model; generative AI is constrained to draft explanation. SQLite remains useful for local/test work, while PostgreSQL and private object storage are supported for hosted operation. These choices increase implementation complexity but better match an evidence-centered MRV workflow than the original single-user prototype.

8. Component-Level Design

8.1 Persistent Data Sources and Interfaces

The primary transactional store is accessed through SQLAlchemy and supports SQLite for local/test use and PostgreSQL for hosted use. Private evidence attachments and managed-model artifacts can use local filesystem storage or a private S3-compatible backend. Satellite observations are retained in versioned time-series cache tables; methodology source documents are indexed into document/chunk records; background jobs and worker heartbeat state are

persisted in the same database-backed queue. These sources are accessed through application modules rather than by the frontend directly.

8.1.1 Context-Level Data Flow (DFD Level 0)

At Level 0, Terra-Audit is treated as one process. The diagram shows the external actors/services and major information exchanges only; internal data stores and module-level processes are deferred to Level 1.

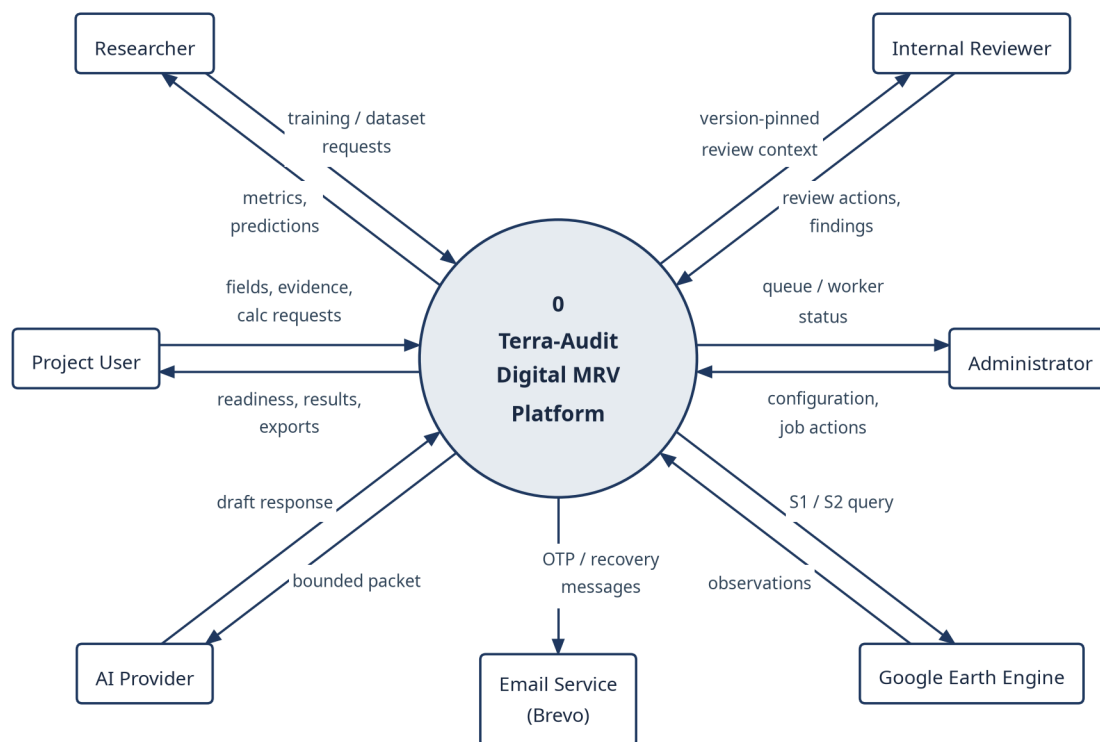


Fig 15: Level-0 data-flow diagram: external entities and the information they exchange with Terra-Audit.

8.1.2 Level-1 Data Flow (DFD Level 1)

Level 1 decomposes the context process into five implementation-backed processes and five logical data stores while preserving the external flows of Fig 15.

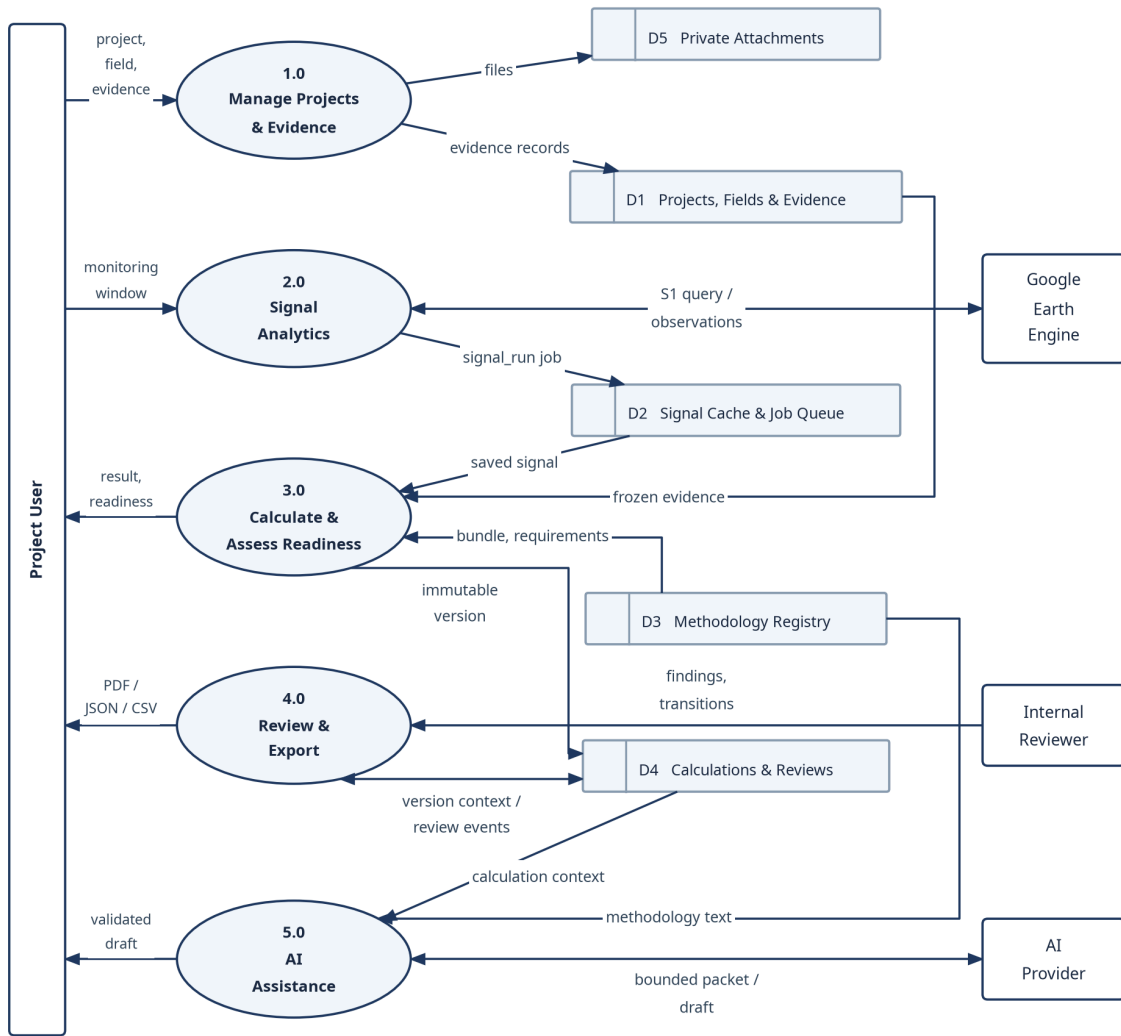


Fig 16: Level-1 data-flow diagram: processes, data stores and external flows.

8.2 API Design

The backend exposes 24 FastAPI routers. All routes except authentication require a bearer JWT; the organization is derived from the token, never from the request. Table 12 groups the representative endpoints.

Table 12: REST API summary by router group

Router group	Representative endpoints	Purpose
Auth & accounts	POST /auth/login, GET /auth/me, POST /auth/register/request-otp, /auth/recovery/*	Sign-in, OTP registration, recovery
Team	GET/POST /team/users, POST /team/invitations	Organization users and invitations
Fields & farms	GET/POST /fields, PATCH /fields/{id}, POST /fields/parse/{geojson kml upload coordinates}	Field registration and geometry
Projects	/projects, /projects/{id}/members, /projects/{id}/fields	Projects, roles, dated memberships
Evidence	/fields/{id}/crop-seasons/..., /attachments, /fields/{id}/soil-*, /production-records	Seasons, observations, files, soil and production

Router group	Representative endpoints	Purpose
Signals & monitoring	POST /fields/{id}/signal-runs, GET /signal-runs/{job_id}, /projects/{id}/monitoring/bulk-run	Sentinel analytics and batches
Calculations	POST .../calculations/preview, POST /fields/{id}/calculations, GET /calculations/{id}/chain	Preview, commit, history
Methodology	/methodology/bundles, /projects/{id}/methodology-applicability, /methodology/corrections	Registry, applicability, corrections
Reviews	/projects/{id}/submissions, /submissions/{id}/transition, /submissions/{id}/findings	Internal review workflow
AI	POST /projects/{id}/ai/explain, POST /ai/train, POST /ai/dataset/build	Explanations and ML
Export	GET /calculations/{id}/evidence/pdf, /fields/{id}/verifications/{vid}/evidence/{pdf json csv}	Evidence packages
Operations	GET /admin/queue-status, /admin/product-readiness, /portfolio/summary, /health	Monitoring and health

8.3 Behavioral Design

Behavioral design combines an event table (Table 13), the persisted state machines (Fig 17–Fig 19) and sequence diagrams of the three highest-risk workflows (Fig 20–Fig 22).

8.3.1 Behavioral Event Table

Table 13: Behavioral event table

Event	Trigger / precondition	System response	Resulting state
Signal run requested	Project user; rice_awd field and window	Return compatible cache or enqueue a deduplicated signal_run; worker fetches Sentinel-1 and stores the result	pending → running → done / error / cancelled
Calculation committed	Writer role; valid inputs; new Idempotency-Key	Freeze snapshot, run engine, evaluate readiness, persist version	draft or ready_for_review; prior version superseded on correction
Review submitted	Calculation is ready_for_review	Create version-pinned submission and review event	submitted
Review action	Assigned reviewer; current if_version	Record finding/comment or allowed transition; open blocking finding prevents approval	in_review / changes_requested / internally_approved / rejected / withdrawn
Data-quality issue	Monitoring run detects a problem	Upsert open issue; acknowledge or resolve with actor and reason	open → acknowledged → resolved
AI explanation requested	Provider permission; allowed action	Return cached validated draft or enqueue ai_explain	pending → running → done / error

8.3.2 State Diagrams

Fig 17 shows the durable job lifecycle including lease-based reclaim and bounded retry. Fig 18 shows that calculation versions are never edited in place—a correction creates version N+1 and supersedes the prior version—and the data-quality issue lifecycle. Fig 19 shows the server-enforced internal review transition table.

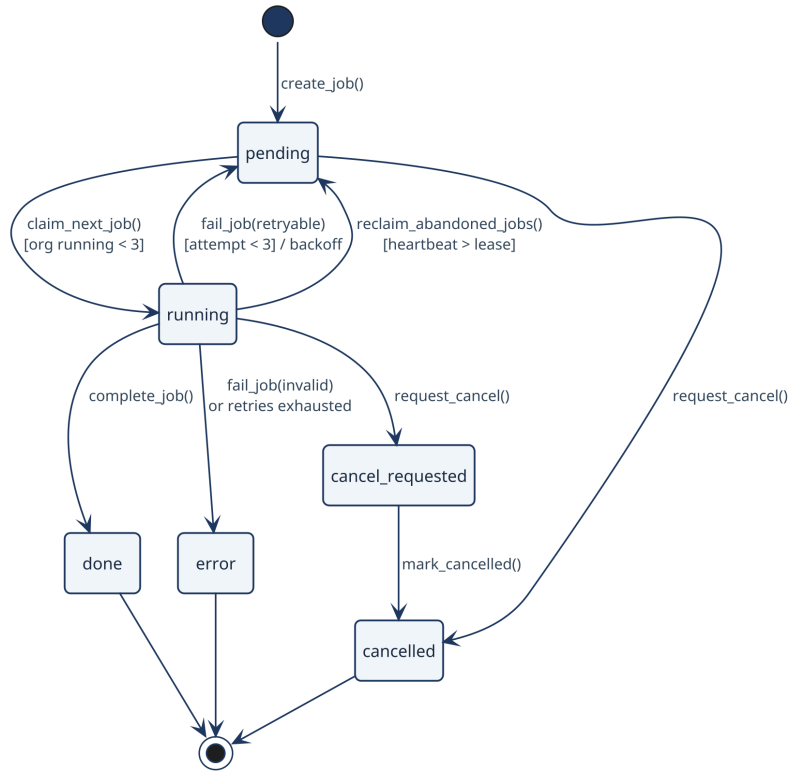


Fig 17: State diagram — durable background job lifecycle (src/jobs/queue.py).

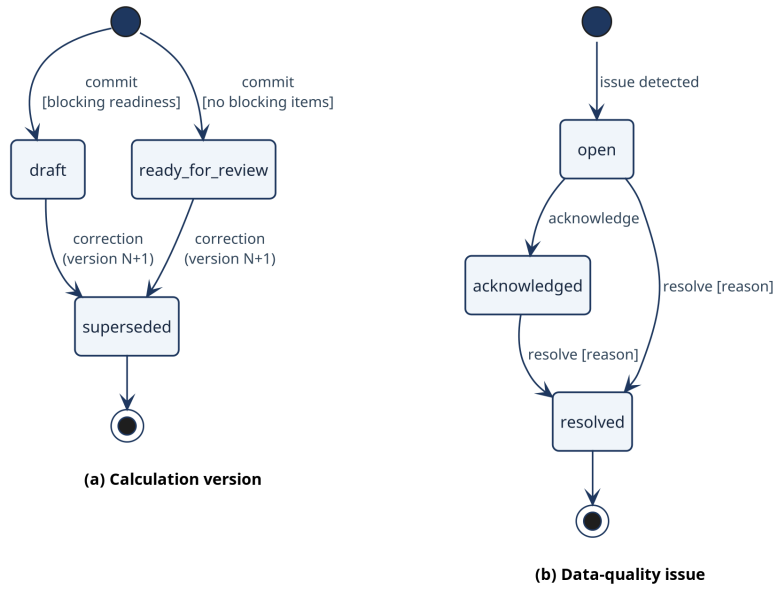


Fig 18: State diagrams — (a) calculation version lifecycle and (b) data-quality issue lifecycle.

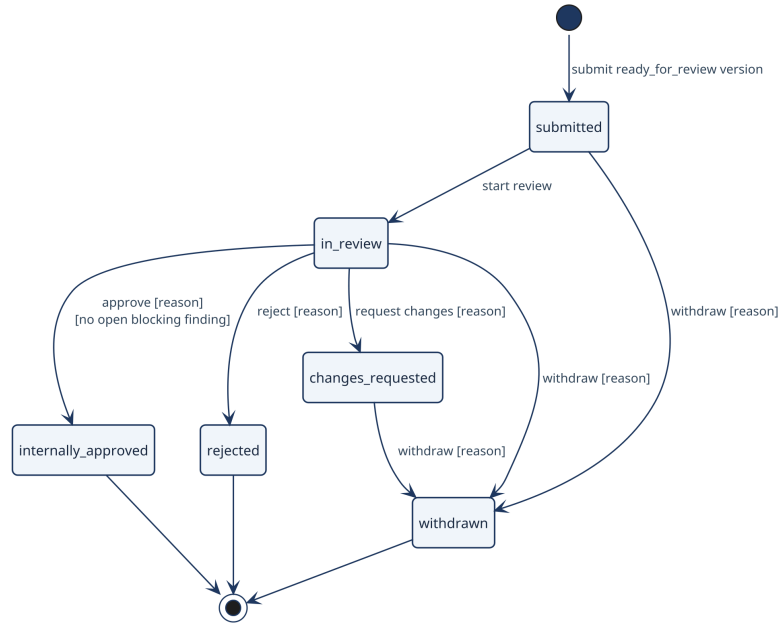


Fig 19: State diagram — internal review submission lifecycle (src/projects/reviews.py).

8.3.3 Sequence Diagrams

The sequences cover the immutable calculation commit, the durable signal run and the AI explanation job; the internal-review interaction is given by the readiness/review activity diagram (Fig 6) and the review state machine (Fig 19). Participants are the implemented Next.js, FastAPI, domain and worker boundaries.

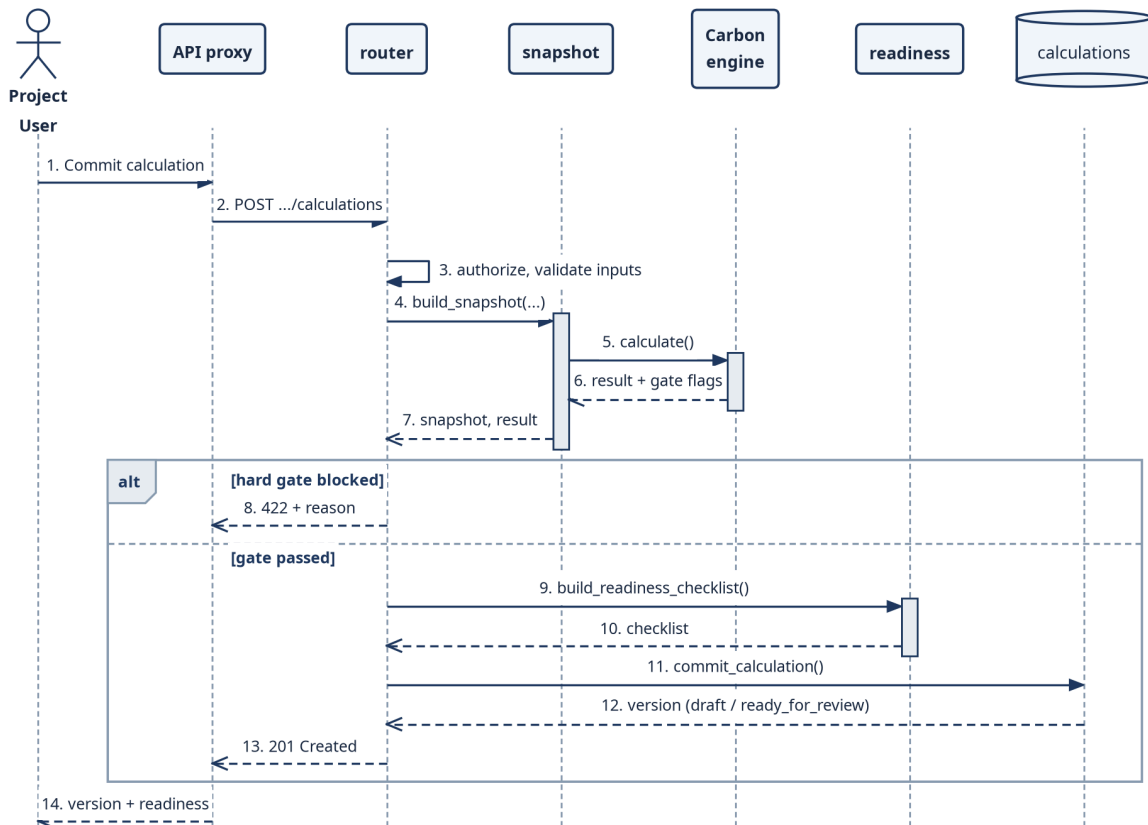


Fig 20: Sequence diagram — calculation commit from authenticated request to immutable version.

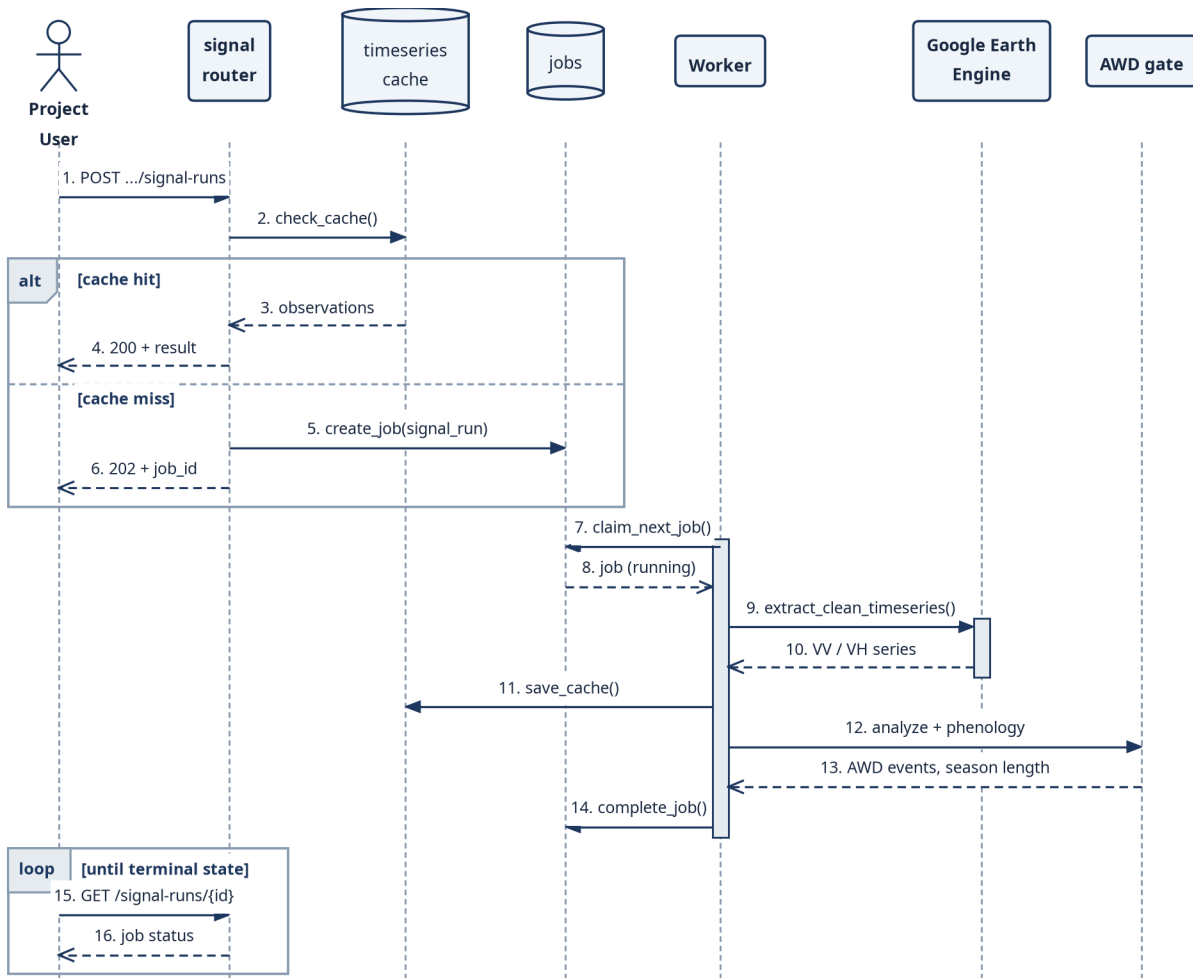


Fig 21: Sequence diagram — Sentinel-1 signal run through the durable job queue.

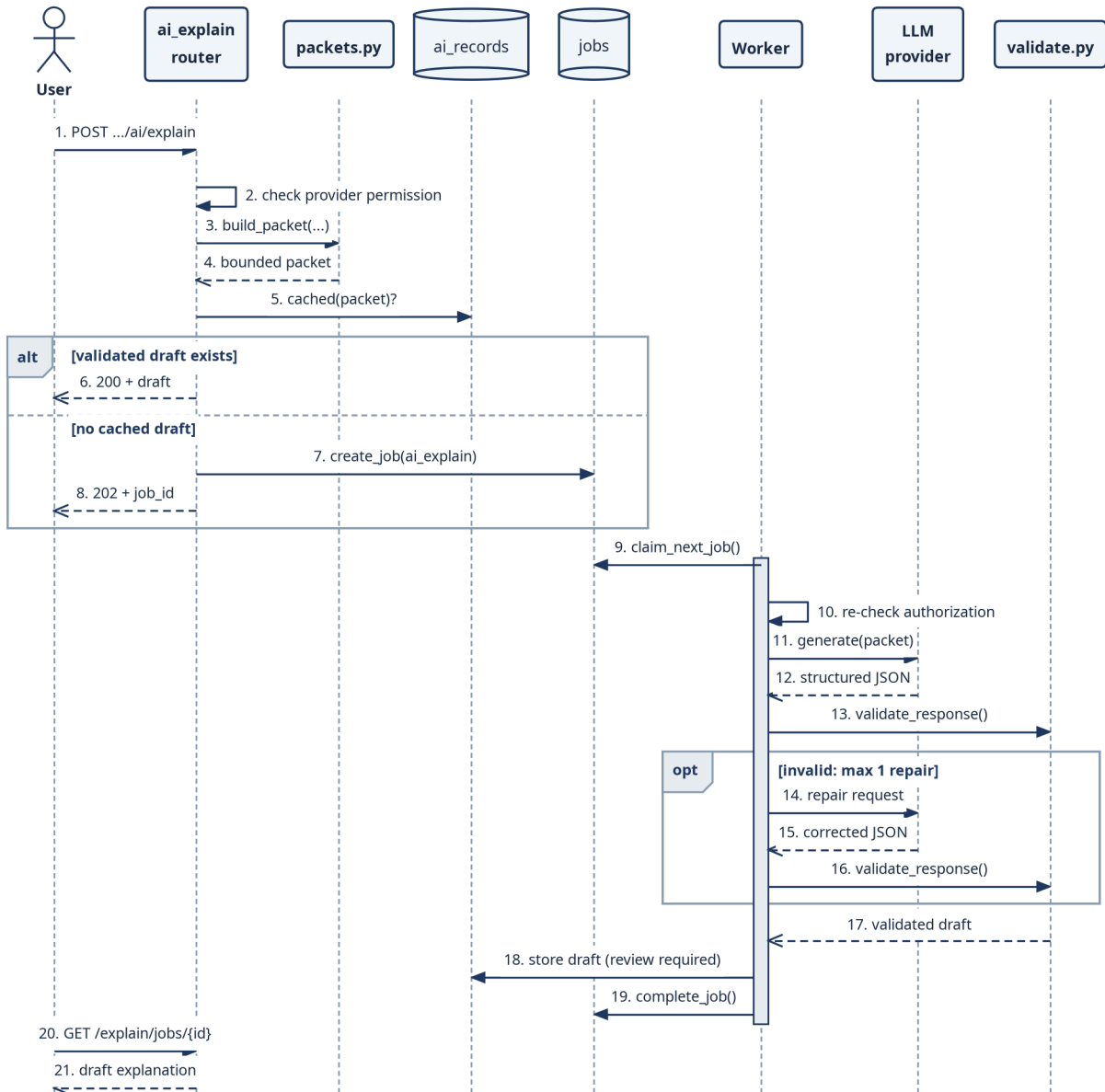


Fig 22: Sequence diagram — bounded AI explanation with server-side validation.

8.4 Deployment Design

Fig 23 shows the deployment architecture. The Next.js frontend is deployed on Vercel and calls the FastAPI backend over HTTPS; the documented production backend runs on Render, and the repository also provides docker-compose (backend, worker and frontend services) and a Railway configuration. Backend and worker run from the same Python 3.12 image: uvicorn serves backend.main:app with a /health endpoint, while python -m backend.worker claims durable jobs from the shared database and calls Earth Engine and AI providers. PostgreSQL is used when DATABASE_URL is set and SQLite otherwise; attachments and model artifacts use local storage or an S3-compatible bucket.

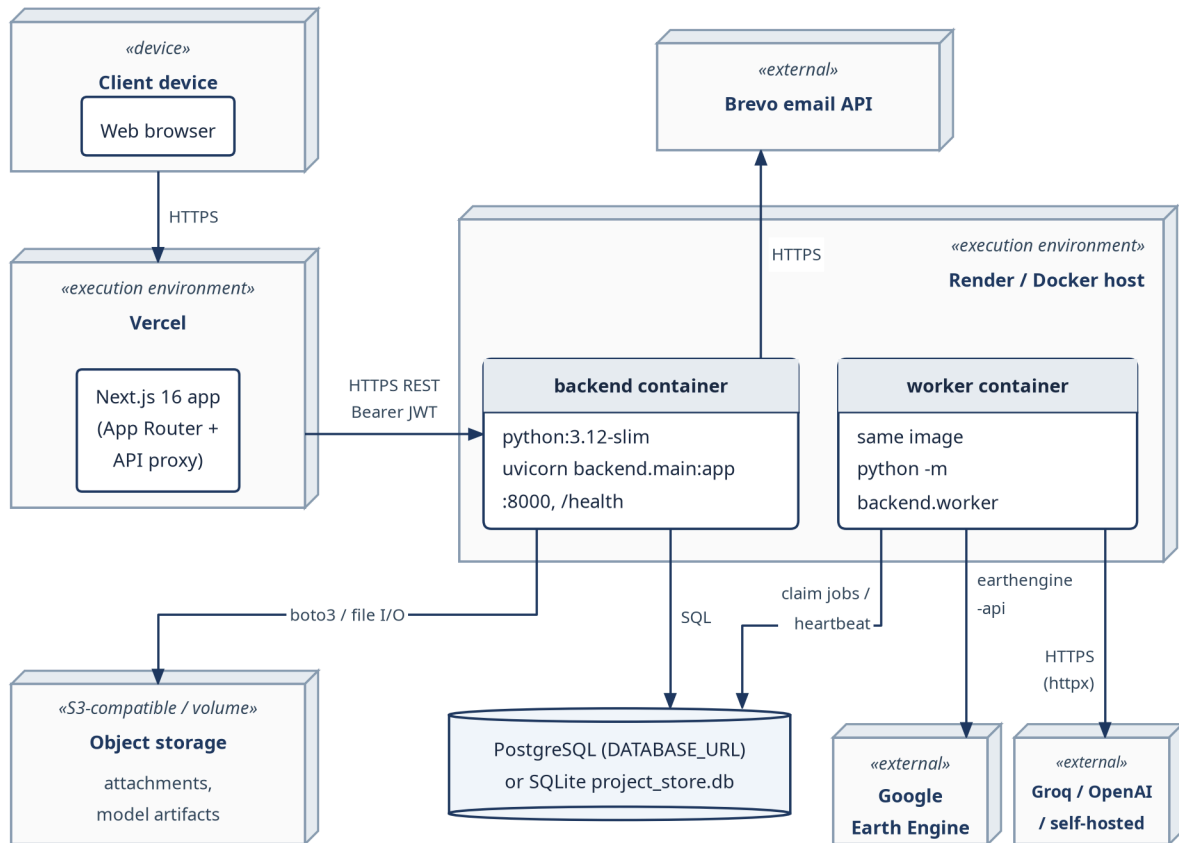


Fig 23: UML deployment diagram of Terra-Audit.

9. AI Engineering Design

9.1 Classical AWD ML Pipeline

The classical AWD ML pipeline applies to the rice signal path and remains experimental/supporting analytics rather than methodology-authoritative evidence:

- DatasetBuilder constructs training rows from cached rice time-series and uses threshold-gate labels; the label source is therefore not independent ground truth.
- Feature engineering builds a tabular matrix from satellite/time-series attributes while excluding the gate's own decision flags as direct predictor features.
- The shared training harness trains Random Forest (200 trees, class-balanced) and XGBoost (200 estimators, max depth 4) using field-grouped GroupKFold evaluation; at least two independent fields are required and each training fold must contain all label classes.
- Persisted model bundles store the model, classes, feature names, satellite processing version, and the field-grouped split strategy. Loading rejects stale processing versions so models are not silently applied to incompatible satellite features.
- Evaluation reports pooled out-of-fold threshold-agreement metrics such as precision/recall/F1, confusion matrices, feature importance, and one-vs-rest ROC data. The reporting code deliberately avoids presenting these values as independently verified irrigation accuracy.

9.2 Model Selection and Evaluation

Random Forest and XGBoost remain appropriate baselines because the current AWD features are a comparatively small structured tabular set rather than raw imagery. Both can model nonlinear feature interactions, are inexpensive to retrain, and expose feature importance. The final implementation evaluates them with the same field-grouped split strategy so model comparison is not inflated by placing observations from the same field in both training and validation folds.

Field grouping fixes an important evaluation-leakage problem from the earlier design, but it does not fix label provenance. Because the target labels are still generated by the threshold detector, model metrics measure how closely the models reproduce that detector. Independent water-level or irrigation observations are still required before the ML results can support a claim of real-world AWD event accuracy.

9.3 Managed Crop Models

The final system also contains managed crop-model workflows for exploratory crop classification/benchmarking. These models use reviewed crop-identity labels and held-out grouping, but they remain research features rather than carbon-accounting authority. Their presence broadens the AI workspace beyond AWD classification without changing the deterministic VM0051/VM0042 calculation boundary.

9.4 Generative AI Explanation Architecture

Generative AI is implemented as a bounded explanation workflow rather than as an accounting engine. An authenticated project request selects an allowed explanation action, after which the server builds a deterministic action-specific context packet from current project/evidence state and bundle-scoped methodology text. The request is served from cache when an equivalent validated explanation exists; otherwise a durable `ai_explain` job is queued and processed by the worker. Supported provider adapters include fake, self-hosted, Groq, and OpenAI.

The worker rechecks authorization, provider configuration, and evidence context before calling the provider. The structured response is then validated by server code and may receive at most one repair attempt. A final authorization/evidence check is performed before the explanation is persisted and shown as a draft requiring human review.

9.5 Prompt and Context Design

Prompting is therefore applicable in the final system, but the design is intentionally constrained by server-owned context. Packets contain the requested action, active methodology bundle, selected methodology chunks and linked corrections, evidence/calculation context, sentence identifiers, allowed requirement/record identifiers, hashes, and server-owned limitations/next actions. The model is not permitted to invent its own authoritative requirement identifiers or alter official calculation values. Retrieval is bundle-scoped so explanations are grounded in the methodology version/corrections selected by the application rather than in unrestricted general knowledge.

The output contract is structured rather than free-form only. Validation checks schema compliance, supplied sentence/requirement/record membership, citation presence, numeric/reference support, correction handling, prohibited claims, and server-owned

actions/limitations. These checks improve structural reliability but do not prove that every cited sentence semantically entails the generated claim.

9.6 AI Authority Boundary and Evaluation Limitations

AI features are deliberately outside the accounting authority boundary. Random Forest/XGBoost and managed crop models are experimental analytics; generative providers produce draft explanations only. Deterministic code owns methodology readiness, carbon equations, persistence gates, and official numeric outputs. No AI component can turn a blocked requirement into compliant status, perform external VVB verification, or create registry-issued credits.

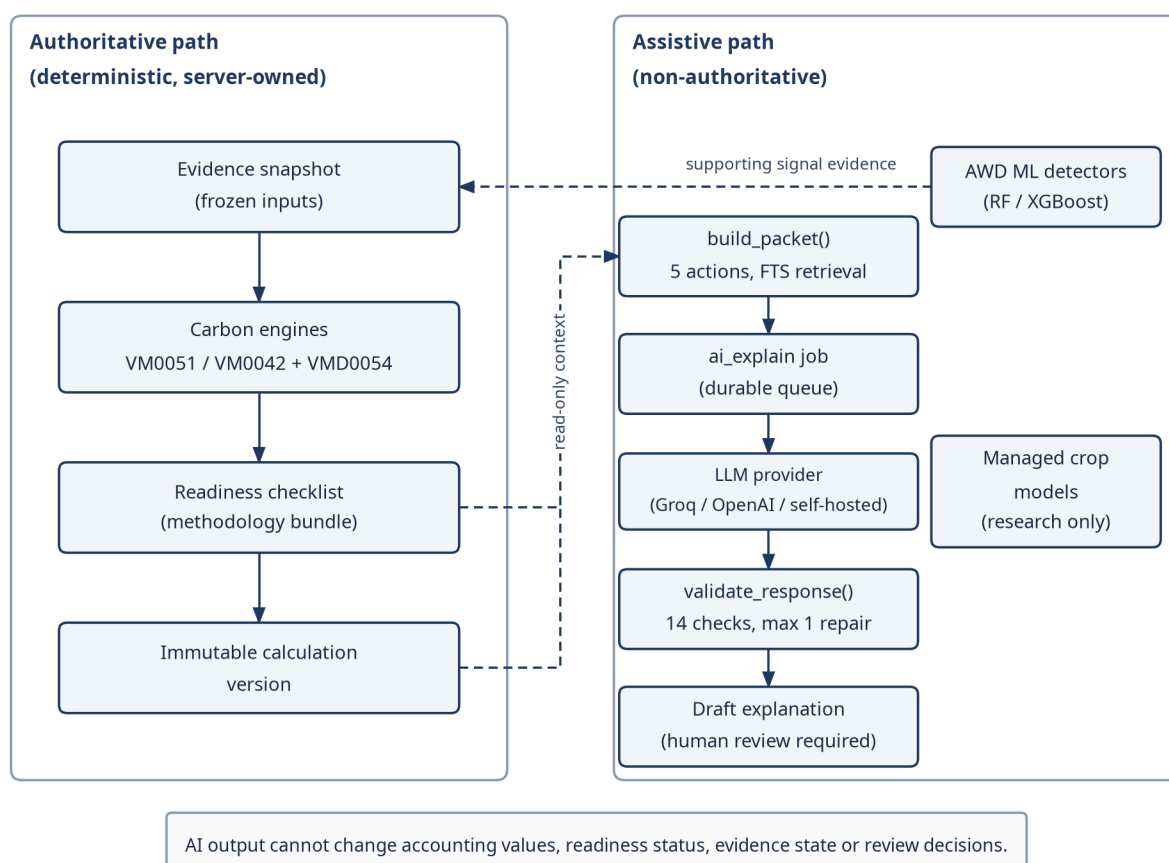


Fig 24: AI authority and data flow: deterministic, server-owned accounting path versus the non-authoritative assistive path.

The software regression suite covers the AI code paths, but the dedicated 20-case generative-AI evaluation corpus remains incomplete because the canonical packet fixtures are missing. Likewise, no independent AWD ground-truth dataset has been completed. These gaps are reported as evaluation limitations rather than hidden behind software-test success.

10. Interface Design

The final interface is a responsive Next.js web application centered on authenticated project context, explicit permissions/readiness, durable-job status, versioned calculation history and internal review. Queued, running, blocked, failed, review-required and completed states remain visibly distinct rather than being collapsed into a generic success/failure message.

10.1 Interface Objects and Actions

Interface objects and their actions were derived from the user and task analysis of the SRS. Table 14 lists each object, its operations and the route that hosts it. Actions are role-aware, but server-side authorization remains authoritative.

Table 14: User-interface objects and actions

Interface object	Actions (operations)	Route
Session / account	Sign in, register with OTP, recover access, sign out	/login, /register, /account-access
Project	Create, edit, add members, assign fields, view dashboard	/projects/[projectId]
Field	Register geometry (draw, GeoJSON, KML, coordinates), edit metadata	/fields/new, /fields/[fieldId]/edit
Crop season & evidence	Create season, correct version, add observation / practice event, upload file	/fields/[id]/crop-seasons, /evidence-files
Soil & production evidence	Add sampling plan, sample, lab result, custody event; import production	/fields/[id]/soil-evidence, /production-records
Signal analytics	Choose window and detector, start run, cancel, inspect series	/fields/[id]/signal-analytics
Calculation	Preview, commit (freeze evidence), view version chain, export	/fields/[id]/calculations, /ledger
Methodology readiness	View bundle, applicability and requirement status; record determination	/projects/[id]/methodology
Internal review	Submit, assign reviewer, transition, add finding / comment	/reviews, /reviews/[submissionId]
AI workspace	Request explanation, train / evaluate model, predict	/projects/[id]/ai, /ai-validation
Operations	View queue and worker status, retry / cancel batch, setup	/admin/queue, /admin/setup

10.2 Primary User Workflows

The core user journey is: select project/field → build or review evidence → run rice signal analytics when applicable → preview and save a versioned calculation → inspect readiness → submit an eligible version to internal review → export an available artifact. Researchers use signal/ML/AI workspaces; administrators manage methodology sources, provider permissions and worker/job state.

10.3 UI Events and State Transitions

Table 15 maps the most important user events to the visible interface state and the server-owned state behind it (see the state diagrams in Chapter 8).

Table 15: UI events and resulting state changes

User event	UI state before	UI state after	Server-owned state
Start signal analysis	Idle form	Queued → running progress → result chart or error banner	signal_run job pending / running / done / error
Cancel running job	Running	Cancelling → cancelled	cancel_requested → cancelled

User event	UI state before	UI state after	Server-owned state
Preview calculation	Input form	Preview result with readiness badges	None (no persistence)
Commit (freeze evidence)	Preview shown	Committed version (draft / ready for review) or blocking message	New calculation version
Submit for review	ready_for_review version	Submission listed as submitted	review_submissions = submitted
Reviewer transition / finding	In review	Status chip and finding list update; approval refused while a blocking finding is open	review_events appended
Request AI explanation	Idle	Pending → validated draft labelled for human review	ai_explain job; ai_records draft
Acknowledge / resolve issue	Open issue	Acknowledged / resolved with actor and reason	data_quality_issues status
Forbidden or invalid action	Any	Inline error message; entered values kept	403 / 422; no state change

10.4 Responsive, Validation and Accessibility Design

The frontend uses shared labeled controls, reusable validation messages, narrow-screen scrolling/containment, text-plus-color state cues, standardized date/number formatting and keyboard-aware shared controls. These improve usability across desktop/tablet/mobile layouts, but no dedicated final browser-E2E or formal accessibility suite was independently executed.

10.5 Interface States as Seen by the User

Fig 25–Fig 29 show representative states of the web interface using clearly labelled synthetic demonstration records: project navigation, monitoring, calculation readiness and inputs, review views and the AI workspace. Captions describe only what is visible.

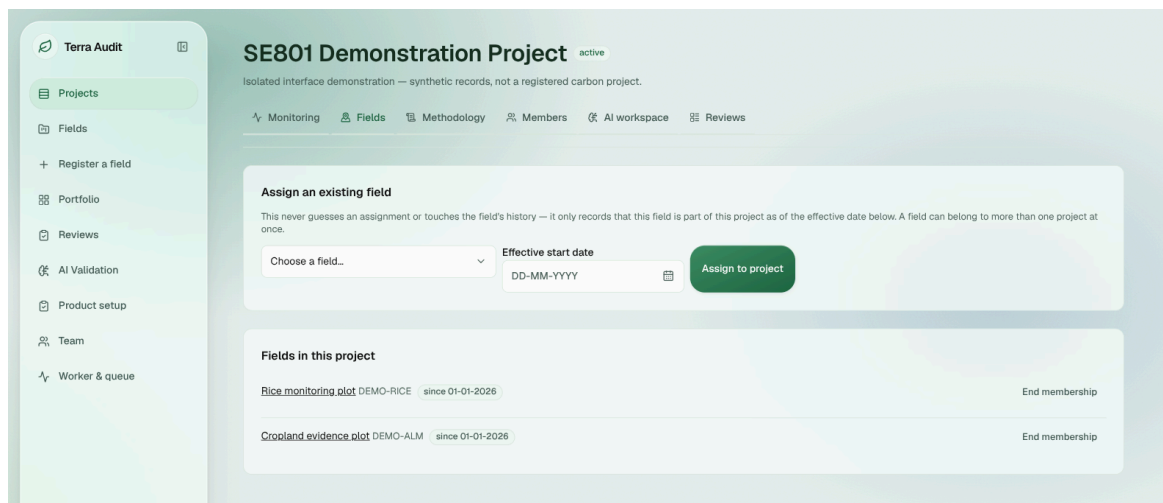


Fig 25: Application shell and project navigation: sidebar, project tabs, field-assignment controls and two demonstration fields.

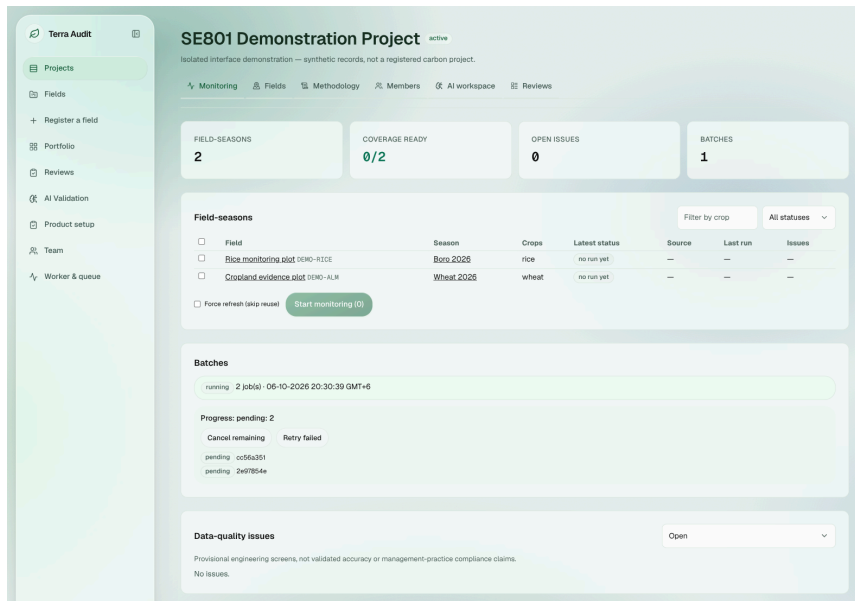


Fig 26: Monitoring batch state: one running batch with two pending child jobs; no data-quality issues.

(a)

Readiness checklist

This reflects what this implementation can check automatically, plus any recorded expert determinations — it is not a certification of full methodology compliance.

needs review

Explain

common.methodology_applicability

Declared crop(s) [rice] are within pathway 'vm0051_rice_awd's usual scope per crop taxonomy alone — this is an indicative signal, not a full applicability determination. Actual land-use conditions (e.g. water regime, drainage, prior land use) must still be confirmed by a reviewer before this can be marked satisfied.

Required evidence: Field type (immutable at registration) plus the crop(s) declared in linked crop seasons must be within the selected pathway's documented scope.

satisfied

common.monitoring_period_coverage

Linked crop seasons cover the requested monitoring period.

Required evidence: Crop-season start/end dates spanning the monitoring period.

needs review

Explain

common.evidence_review_status

1 of 1 field observations are not yet accepted by an independent reviewer.

Required evidence: Each field observation accepted by a team member other than its author.

missing

Explain

vm0051.required_measurement_inputs

AWD event count and season length are the engine's required satellite-derived inputs.

VM0051: Improved Management in Rice Production Systems v1.1 §8.2.3 Eq. 6

Required evidence: AWD event count and season length from signal analytics or manual entry.

Cannot be manually overridden.

needs review

Explain

vm0051.qa3_pathway_project_size

Run a calculation preview to evaluate the QA3 project-size gate.

VM0051: Improved Management in Rice Production Systems v1.1 §8.6.3

Required evidence: Computed from the calculation result; not evidence supplied by a user.

Cannot be manually overridden.

unsupported

Explain

vm0051.leakage_assessment

Not implemented — no evidence can satisfy this in the current codebase.

VM0051: Improved Management in Rice Production Systems v1.1 §8.4

Required evidence: Not implemented — no evidence can satisfy this in the current codebase.

Implementation: not implemented by this system

Cannot be manually overridden.

(b)

Engine inputs

Manual evidence entry for this monitoring period. No measurements are assumed. Use zero only when the evidence records zero; amendment rates may be zero when no material was applied.

Use saved analytics

Only runs matching this field, registered area and both monitoring dates are offered. Confirm that the boundary and selected crop seasons match: older signal jobs do not freeze those versions. Nitrogen, water regime and amendments remain manual.

No compatible run among recent completed analytics. Run analytics for 01-01-2026 to 31-05-2026, or enter documented values manually.

Keep values as manual entry

AWD events (evidence-backed)

Season length (days)

N input (kg N/ha)

Pre-season water regime

Baseline amendment type

Baseline amendment rate (t/ha)

Project amendment type

Project amendment rate (t/ha)

Field area used is always this field's registered area_ha (1.14 ha) — it is frozen from the field record, not re-entered here.

Correct an existing calculation (optional)

☐ I have checked these values against evidence for the selected period.

Preview calculation

Commit (freeze evidence)

Fig 27: Rice calculation readiness and inputs: (a) readiness items marked needs review, satisfied, missing or unsupported; (b) the evidence-backed input form with Preview and Commit (freeze evidence).

(a)

Reviews

Internal review only — this is not external verification or registry issuance.

Notifications
No notifications.

My reviews

Nothing assigned to you
Submissions assigned to you for review will appear here.

Project review queue

SEBOI Demonstration Project

All statuses

- All statuses ✓
- submitted
- in review
- changes requested
- internally approved
- rejected
- withdrawn

No submissions
No submissions have been submitted for review in this project yet.

(b)

Record an evidence review

Project leads and administrators can decide reviewable requirements. Decisions apply to the selected project, dates and current evidence. Changed leakage inputs or production records require a new review.

common.methodology_applicability

Further review needed

Demo evidence requires independent confirmation of water regime and prior land use. No applicability approval has been granted.

Record review

Calculation history

draft v1 06-10-2026 20:30:39 GMT+6 Estimate: 1.4627 tCO2e

Explain why blocked What changed? Download JSON

Fig 28: Review-related states: (a) the internal Reviews page with its status filter; (b) an evidence-review form and a draft calculation estimate.

Terra Audit

New Project planning

ABC

Monitoring Fields Methodology Members AI workspace Reviews

Multi-crop AI supports monitoring and evidence preparation. Model scores are uncalibrated; outputs require human review and do not authorize carbon credits.

AI provider
grop openal/gpt-oss-20b. Configuration presence does not verify worker connectivity. Drafts still require human review.

Train a crop model
Requires at least four eligible field-seasons, two crops, and independently reviewed labels. Training uses only active project fields and reserves independent groups for evaluation.

0 eligible seasons 0 crops 1 excluded seasons

Version name Model Hold out by

September crop baseline Random forest Field

Train and evaluate

Model versions
No trained versions yet.

Active model Disable predictions Minimum model score 0.8

Reason for activation or change

Save activation decision

Current revision: 0; threshold: 0.8. Activation records your decision; it does not establish regional validity.

Activation history

Fig 29: AI workspace — crop-model controls: provider label, eligibility count and training/evaluation controls.

11. Implementation

This chapter summarizes the final implemented software rather than reporting midterm progress. Status labels distinguish implemented capability from partial methodology coverage, experimental research features, and validation/operational gaps. A green software test suite does not by itself convert a partial or experimental feature into a fully validated MRV method.

11.1 Final Feature Status

The status of every functional requirement—implemented, partial or experimental—is reported once, in the requirements traceability matrix (Table 19), together with its verification evidence. “Implemented” means the capability exists and is covered by the architecture; “Partial” means scope, provenance, methodology, authorization or operational limits remain; “Experimental” marks research features that must not be presented as validated accounting evidence.

11.2 Worked Calculation Checks

Rice AWD fixture — Inputs: 2 AWD events, 120-day season, 1.0 ha, 100 kg N/ha, short pre-season water regime, and the default 5 t/ha straw amendment in both scenarios. The engine gives $SC_o = (1+5)^{0.59} = 2.878122$; baseline CH₄=293.568470 kg and project CH₄=161.462659 kg, so avoided CH₄=132.105812 kg=3.698963 tCO₂e. The QA3 15% deduction is 0.554844 tCO₂e, leaving 3.144118 tCO₂e; the irrigation N₂O term is 0.083210 tCO₂e. Net deterministic estimate=3.060908 tCO₂e, below the 60,000 tCO₂e/year QA3 gate. This is a code-level worked fixture, not an issued VCU.

Cropland ALM fixture — Inputs follow the fixed 5 ha, one-year regression scenario: baseline synthetic N 120 kg/ha, residue burning 500 kg/ha and diesel 40 L/ha; project synthetic N 80 kg/ha, organic N 20 kg/ha, no residue burning and diesel 25 L/ha; SOC cells are project start [30,32,29], project final [34,35,33], control start [30,31,29], control final [30,31,30]. The mean project-minus-control SOC change is 3.333333 tCO₂e/ha; the independently reconstructed uncertainty is 16.115318%. The engine regression fixture gives ER=1.073643 tCO₂e and uncertainty-adjusted CR=13.980780 tCO₂e; with a 15% non-permanence deduction on the removal term, the deterministic estimate is 12.957306 tCO₂e. The fixture still discloses the separate other-leakage limitation and does not imply external verification or issuance.

Remaining engineering and validation gaps—browser end-to-end tests, PostgreSQL migration, live external services, independent AWD ground truth and external VVB verification—are consolidated with the limitations in Chapter 14.

12. Testing

12.1 Testing Objectives

- Verify the deterministic VM0051 v1.1 and VM0042 v2.2 scoped accounting engines against their implemented factors, equations, golden values, edge cases, and hard numerical gates.
- Verify methodology/readiness behavior so unsupported, incomplete, stale, reviewer-dependent, or hard-blocking conditions remain explicit and cannot silently become satisfied or zero.
- Verify versioned calculation/snapshot behavior: later evidence, corrected seasons, recalculation, or supersession must create new state without silently rewriting the historical numerical inputs/results of an existing calculation version.

- Verify authentication, organization isolation, project-role checks, private evidence access, account recovery, and durable job/worker state transitions across protected backend workflows.
- Verify frontend contracts through automated tests, linting, TypeScript checking and a production build, and verify that backend changes do not break request/response schemas relied on by the Next.js application.
- Keep software verification distinct from scientific and external assurance: passing tests do not establish independent AWD field accuracy, production security certification, PostgreSQL migration correctness, live provider reliability, or VVB/Verra validation and issuance.

12.2 Testing Approach, Strategy and Pass/Fail Criteria

Testing is layered so that a green arithmetic engine alone cannot hide a failure in authorization, evidence persistence, background processing, or frontend contracts. Unit and regression tests cover deterministic calculation logic and helpers; API/integration tests exercise authentication, tenant scoping, evidence, methodology, calculation, export, and job workflows against isolated databases; frontend tests, lint, TypeScript, and production build checks validate the web layer; and manual/operational checks cover deployment and external-service concerns that are not reproduced by the isolated suite.

A software verification item passes when the expected behavior completes without an unexpected failure in the final verification. For the regression suite this means zero failed backend tests, zero failed frontend tests, a successful production build and TypeScript check, and no ESLint errors. A methodology or scientific claim has a higher bar: an automated test can prove that code follows a specified equation or gate, but it cannot prove methodology completeness, independent AWD event accuracy, production security certification, or external VVB/Verra acceptance.

Test environment: Ubuntu Linux with Python (backend virtual environment, pytest 8) and Node.js 22 for the frontend. Backend tests run against isolated temporary SQLite databases with FastAPI's TestClient, fixed fixtures and stubbed external providers, so results do not depend on Earth Engine, AI or email availability.

12.2.1 API Endpoint Verification

The backend suite contains direct FastAPI TestClient coverage for the endpoints in Table 16. Only endpoints with HTTP-level tests are listed; lower-level domain tests are not relabelled as API tests.

Table 16: Direct API endpoint verification

ID	Endpoint	Verified behavior	Test evidence
API-01	POST /auth/login	200 + token for valid credentials; 401 for wrong/unknown credentials.	test_auth_endpoints.py
API-02	GET /auth/me	200 with authenticated organization/role/email; 401 without a token.	test_auth_endpoints.py, test_registration.py
API-03	POST /fields	201 for authorized admin; 403 viewer; 409 duplicate ID; 422 invalid field type.	test_fields_endpoints.py, test_field_type_guards.py
API-04	GET /fields/{field_id}	200 for owning tenant; 404 for another organization.	test_fields_endpoints.py,

ID	Endpoint	Verified behavior	Test evidence
			test_field_type_guards.py
API-05	POST crop-season / GET evidence	201 authorized create; 403 viewer; 422 invalid payload; 404 cross-tenant evidence access.	test_multicrop.py
API-06	POST monitoring-run / GET job	202 durable job, pending before worker dispatch, done after execution, 404 cross-tenant.	test_multicrop.py
API-07	POST /fields/{field_id}/signal-runs	200 on compatible local cache hit; 202 durable job on cache miss.	test_signal_run_cache_path.py
API-08	POST /fields/{field_id}/calculations	201 with Idempotency-Key and valid ALM context; persists immutable calculation/snapshot state.	test_evidence_verification.py
API-09	GET /calculations/{calculation_id}/evidence/pdf	200 application/pdf for owner; 404 cross-tenant, 403 unauthorized viewer; PDF remains snapshot-backed after live evidence changes.	test_evidence_verification.py
API-10	GET/POST methodology corrections	readable list; 403 viewer confirm; 200 admin confirm; 404 missing resource.	test_methodology_corrections.py
API-11	GET/POST /team/users	403 non-admin; 200 admin list; 201 invite; 409 duplicate email; 422 invalid role.	test_team_router.py

Review, batch, AI-provider and other behavior is also covered in the wider regression/integration suite, but it is not presented here as direct endpoint evidence unless a TestClient case was located. Overall regression status is given in Table 18.

12.3 Detailed Test Cases

Table 17 expands representative automated tests into the test-case format with detailed outcomes. All twelve cases are part of the backend suite re-run on 7 October 2026.

Table 17: Detailed test cases with outcomes

ID	Test case / precondition	Steps / input	Expected result	Actual result	Status
TC-01	VM0051 QA3 size gate. 2 AWD events, 120-day season, 500,000 ha, 0 kg N	CarbonAssetEngine.calculate_credits()	Reject results above 60,000 tCO2e/yr; no final quantity	qa3_pathway_valid=False; final_issuance=None; reason cites the 60,000 threshold	PASS test_carbon_calculator.py
TC-02	VM0042 without leakage evidence. Valid 5 ha ALM fixture, no frozen leakage_result	AlmCarbonEngine.calculate_credits()	Block instead of assuming zero leakage	production_decline_leakage_blocked=True; final_issuance=None; not issuable	PASS test_carbon_calculator_alm.py
TC-03	Cross-organization isolation. Field in testorg; admin of otherorg	GET /fields; GET /fields/{id}	Other organization cannot list or read the field	Empty list; HTTP 404	PASS test_fields_endpoints.py
TC-04	Snapshot-backed ALM PDF. Committed ALM calculation	Change live field/practice data; GET /calculations/{id}/evidence/pdf as	Unauthorized rejected; PDF reflects the frozen snapshot	404 / 403 / 200 application/pdf; content matches the snapshot	PASS test_evidence_verification.py

ID	Test case / precondition	Steps / input	Expected result	Actual result	Status
		other org, viewer, owner			
TC-05	Durable monitoring job. Authorized crop season	POST monitoring run; worker claims; query from other org	202 pending → running → done; other org denied	Lifecycle asserted; cross-org 404; stored evidence unchanged	PASS test_multicrop.py
TC-06	AI output validation. Packet allowing one sentence and one requirement	Validate invented values/records and prohibited claims; one-retry path	Invalid output rejected; one repair may succeed	ExplanationValidationError; retry_count=1 succeeds; two failures raise	PASS test_ai_validate.py
TC-07	Idempotent commit retry. ALM field with frozen calculation context	POST /fields/{id}/calculations twice with the same Idempotency-Key	One draft version; retry returns it unchanged	201 then 201 with already_committed=True; 1 calculation row, 1 key row; cumulative state unchanged	PASS test_idempotency.py
TC-08	Signal-run cache path. Rice field; cached and uncached windows	POST /fields/{id}/signal-runs for each window	Cache hit answers synchronously; miss enqueues a job	200 with 20 cached observations; 202 with job_id	PASS test_signal_run_cache_path.py
TC-09	Pathway guards. ALM field and rice field	Signal run on the ALM field; ALM practice schedule on the rice field	Pathway-specific endpoints refuse the wrong field type	HTTP 422 'not satellite-driven'; HTTP 422 'requires cropland_alm_vm0042'	PASS test_field_type_guards.py
TC-10	OTP brute-force lockout. Pending registration	Verify five wrong OTP codes, then the correct code	Lock the registration after five failures	400 for each wrong code; correct code then 429	PASS test_registration.py
TC-11	Role and tenant checks on team management. Analyst, viewer, other-org admin	GET /team/users; POST /team/users	Only admins manage the team; roster is org-scoped	403 for analyst and viewer; other admin sees only own org	PASS test_team_router.py
TC-12	Non-issuable result. Rice result with qa3_pathway_valid=False	commit_carbon_credit_result()	Blocked result is never persisted	NonIssuableResultError; credit history count 0	PASS test_issuance_gate.py

12.4 Verification Results

Table 18: Verification results (7 October 2026, commit bc402f2)

Verification area	Command / evidence	Scope	Pass criterion	Result
Backend regression	python -m pytest -q	Full backend/core suite	0 failed	PASS — 177 passed, 0 failed, 0 skipped; 20 warnings; 117 s
Frontend tests	npm test	node --test suite	0 failed	PASS — 4 passed
Production build	npm run build	Next.js production build	Exit 0	PASS
TypeScript	npm run typecheck	tsc --noEmit	Exit 0	PASS

Verification area	Command / evidence	Scope	Pass criterion	Result
ESLint	npm run lint	Frontend lint rules	0 errors	PASS — 0 errors, 5 warnings
Accounting engines	Backend suite	VM0051, VM0042, VMD0054 gates and golden values	No regression	PASS (in suite)
Methodology / readiness	Backend suite	Bundles, corrections, blocking states	Unsupported stays explicit	PASS (in suite)
Snapshots, auth, jobs, AI validator	Backend suite	Idempotency, tenant guards, queue states, output checks	Covered behavior holds	PASS (in suite)
Generative-AI canonical evaluation	Evaluation runner	20 frozen packet cases	All fixtures run	NOT EXECUTED — fixtures incomplete
Browser E2E / accessibility	—	Full user journeys	No critical defect	NOT EXECUTED
PostgreSQL and live services	—	Schema upgrade; Earth Engine / AI / email; worker recovery	Graceful recovery	NOT VERIFIED
Independent AWD ground truth	—	Detections vs field observations	Event-level accuracy	NOT EXECUTED — no field dataset

All five automated checks were re-run on 7 October 2026 against commit bc402f2 of the development branch. The backend suite reported 177 passed, 0 failed and 0 skipped (20 deprecation warnings, 116.98 s); the frontend tests passed 4/4; the Next.js production build and TypeScript check completed; and ESLint reported 0 errors and 5 warnings. The CI workflow should generate Next.js types before its standalone type check; this ordering issue remains open. Browser E2E, PostgreSQL migration, live external services and independent AWD validation were not executed and are reported as limitations.

12.5 Risks and Contingencies

Earth Engine or Sentinel availability — Signal requests must surface empty-data, authentication, network, or API failures as explicit job/error states. Cached compatible observations may reduce repeat dependency but do not replace provenance or live-service testing.

Worker unavailable or stale — Durable jobs remain persisted rather than disappearing with the API request. Operations views and heartbeat state are used to detect stale workers; supported retry/cancel paths are preferred over pretending completion.

AI provider unavailable or invalid output — Explanation requests remain optional. Cached validated explanations may be reused; otherwise provider/configuration errors, packet-size failures, or structured-validation failures remain visible and no authoritative readiness/calculation state is written.

Database/storage failure — SQLite is suitable for local/test operation and PostgreSQL is supported for hosted operation, but current PostgreSQL migration/restore behavior is not fully verified. Private evidence/model artifacts require authenticated storage access and a backup/restore strategy.

Insufficient satellite observations or weak ground truth — The system can fall back to explicit manual calculation inputs where allowed, but the absence of observations cannot be converted into a scientific AWD claim. Independent field validation remains a separate research requirement.

Methodology incompleteness — Unsupported or unresolved methodology/program requirements remain disclosed as partial, unsupported, reviewer-dependent, or blocking. The contingency is not to weaken a gate for convenience; it is to keep the calculation as draft/limited until the missing requirement is addressed by supported evidence or an external expert/VVB process.

12.6 Requirements Traceability Matrix

Table 19 maps each functional requirement to its use case, implementing component, verification evidence and final status. “Pass within final suite” refers to the 177-test backend run; partial statuses mark implementation or assurance boundaries that remain even when tests are green.

Table 19: Functional requirements traceability matrix

Req.	Use Case / Goal	Component	Verification	Status
FR1	Account access / tenant identity	src/accounts; backend/security.py; auth/account-access routers	Authentication/account- access tests	Automated — pass within final suite
FR2	Project & Field Management	src/projects; field/project/team routers; persistence	Project/team/field regression tests	Automated — pass within final suite
FR3	Signal Analytics	src/signals; src/jobs; signal router/worker handler	Signal/cache/job regression tests	Automated — pass within final suite
FR4	Signal Analytics / AWD research	processing.py; threshold_gate.py; src/ai/ml	Signal and ML regression tests	Implemented; software-tested; scientific validation pending
FR5	Evidence & Season Management	src/evidence/monitorin g.py; attachments/monitoring routers	Evidence/monitoring regression tests	Automated — pass within final suite
FR6	ALM Evidence	src/evidence/soil.py; production.py; ALM schedules	Evidence/ALM regression tests	Automated — pass within final suite
FR7	Carbon Calculation — rice	src/carbon/rice.py; calculation/carbon routers	Rice carbon regression tests	Automated — pass within final suite
FR8	Carbon Calculation — ALM	src/carbon/alm.py; leakage_vmd0054.py; snapshot.py	ALM/leakage regression tests	Automated — pass within final suite
FR9	AI Assistance & Research	src/ai/ml; managed-model modules	ML/model regression tests and code review	Implemented/experi mental; independent accuracy pending
FR10	AI Explanation	src/ai/packets.py; providers.py; validate.py; AI job handlers	AI packet/validator regression tests	Implemented/partia l; canonical 20-case evaluation pending
FR11	Reporting & Export	src/reporting; backend/routers/export .py	Evidence/snapshot/export regression tests	Implemented/partia l; legacy provenance limits remain

Req.	Use Case / Goal	Component	Verification	Status
FR12	History & Portfolio	calculations/version chain; project/history/portfolio views	Calculation/history/portfolio regression coverage	Implemented; final suite green
FR13	Methodology Readiness	src/methodology/registry.py; library/corrections/readiness.py	Methodology/readiness regression tests	Implemented/partial methodology coverage
FR14	Calculation Versioning	src/carbon/snapshot.py; calculations.py; idempotency	Snapshot/version/idempotency tests	Automated — pass within final suite
FR15	Internal Review	src/projects/reviews.py; reviews router/UI	Review-route/domain regression tests	Implemented; entry still depends on readiness blockers
FR16	Portfolio / Operations	portfolio/product-ops/ monitoring-ops views and routers	Backend regression suite + operational review	Implemented/partial
FR17	Authorization & editable metadata	backend/access.py; deps.py; field/project update paths	Tenant/auth/team/field tests	Implemented/partial; project enforcement not uniform everywhere
FR18	Durable Jobs	src/jobs/queue.py; worker.py; job handlers	Job/multicrop/background-work tests	Automated — pass within final suite

13. User Manual, Deployment and Repository

This chapter is the end-user manual for the web application, followed by installation, deployment and repository details. It assumes the account belongs to the required organization and project, and that Earth Engine or an AI provider is configured only for the features that need them. The screenshots in Chapter 10 (Fig 25–Fig 29) illustrate the screens referred to below.

13.1 End-User Workflows

Table 20 gives the step sequence for each end-user task and what the user should check at the end of it. Menu names match the routes in Table 14.

Table 20: End-user workflows

Task	Steps	Check
Sign in and select a project	Sign in, or register with an e-mailed OTP / recover access → open Projects → select or create a project → confirm lead, contributor or viewer membership.	If an action is refused, check the organization, project and role; the backend re-checks every request.
Register a field	From the project, open field registration → enter the field identity and geometry (draw, GeoJSON, KML or coordinates) → choose the pathway rice_awd or cropland_alm_vm0042 → assign the field to the project.	The pathway cannot be changed after registration.
Record evidence	Create crop seasons → add observations, practice events, production records and files; for ALM also soil sampling plans, samples, lab results, custody events, livestock schedules and the leakage assessment.	Use corrections so earlier versions stay visible; never enter zero for missing evidence.

Task	Steps	Check
Rice AWD analysis and calculation	Open Signal Analytics → choose the window and detector → start and wait for completed, failed or cancelled → inspect flooding, drydown and phenology → open Calculations, select the saved signal or enter permitted manual inputs → Preview → Commit (freeze evidence).	Signal output is supporting evidence, not validated ground truth; the internal quantity is not an issued VCU.
ALM (VM0042) calculation	On a cropland_alm_vm0042 field record the evidence above → open Calculations; the server freezes a snapshot and runs VMD0054 leakage and VM0042 → commit a new version → export the snapshot-backed PDF.	Missing applicable leakage evidence stays explicit and blocks the result.
Readiness and internal review	Open Methodology → check the bundle, corrections, applicability and each requirement status → submit a ready_for_review version → the reviewer assigns, records findings and comments, and approves, rejects or requests changes.	Drafts cannot enter review; internal review is not VVB validation or Verra verification.
AI explanation	Choose an action (blocked calculation, missing evidence, applicability, leakage or calculation difference) → submit → read the validated draft when the job completes.	The draft needs human review and cannot change readiness, quantities or evidence.

13.2 Export and Troubleshooting

Table 21: Export options and troubleshooting

Situation	What to do
Export	Versioned calculations export as JSON; snapshot-backed VM0042 versions also export a PDF evidence package; legacy verification records keep their PDF/JSON/CSV packages. Prefer snapshot-backed outputs.
Signal job stays pending	Open Worker & queue status; a stale or absent worker prevents queued jobs from completing.
Earth Engine failure	Check Earth Engine authentication and project configuration, and confirm that the window contains Sentinel-1 observations.
Calculation blocked	Inspect readiness and the pathway hard gates; unsupported requirements cannot be bypassed to obtain a ready label.
AI explanation unavailable	Check provider permission and configuration, the worker state and the reported validation or packet error.
Evidence upload fails	Check file size, type and target; private files are downloaded only through authenticated paths.

13.3 Local Installation and Development Setup

Prerequisites include Python 3.12-compatible tooling, Node/npm, and Earth Engine authentication when the SAR pipeline will be used. A minimal local setup is:

1. Clone the public repository and check out the required revision/branch.
2. Create and activate a Python virtual environment.
3. Install backend dependencies with `pip install -r requirements.txt`.
4. For signal analytics, run `earthengine authenticate` and `earthengine set_project <GCP_PROJECT_ID>`.
5. Create a project-root `.env` with at least the required JWT secret, frontend public URL and any external-service configuration. `DATABASE_URL` is optional; when omitted, local SQLite is used.
6. Install frontend dependencies with `cd frontend && npm install`.
7. Start the API with `uvicorn backend.main:app --reload`.

8. Start the durable worker separately with `python -m backend.worker`.
9. Start the frontend with `cd frontend && npm run dev`.
10. Run the verification commands in Chapter 12 before treating a modified revision as release-ready.

13.4 Deployment and Public Repository

Public source repository: <https://github.com/knah1d/terra-audit>

The development branch contains the code described in this report (commit bc402f2, verified on 7 October 2026). The root README documents the architecture, setup, run and test commands, and configuration examples avoid embedding secrets.

Production uses Vercel for the Next.js frontend and Render for the FastAPI backend; docker-compose and Railway configurations are also provided. PostgreSQL (Neon-compatible) and S3-compatible storage are supported. Hosting is optional for evaluation: the complete application runs locally with the steps in Section 13.3.

A traditional desktop installer is not applicable because Terra-Audit is a browser-based web application. Deployment consists of the web frontend, API, database/storage, and separate worker rather than a downloadable executable installer.

14. Evaluation, Limitations and Conclusion

Terra-Audit has progressed from a local single-user prototype into a multi-domain web application for evidence-centered agricultural MRV preparation. The strongest final outcome is the separation of deterministic accounting/readiness logic from assistive satellite, ML and generative-AI components while adding multi-user access, structured evidence, durable jobs, methodology bundles, calculation versioning and internal review. The software regression results are green, but the project still has clear methodology, scientific, provenance and operational boundaries.

14.1 Final Evaluation

Software architecture — The Next.js/FastAPI/domain/worker structure is materially stronger than the original Streamlit design for multi-user authorization, asynchronous jobs and audit history. Capability-oriented packages keep accounts, projects, evidence, methodology, carbon, signals, jobs, AI, persistence and reporting separate enough that frontend or AI changes do not become calculation authority.

Accounting — The rice and ALM engines are deterministic and regression-tested, with explicit intermediate logic and hard gates. However, both are scoped implementations rather than complete automated implementations of every methodology/program requirement.

Evidence and auditability — Versioned calculation records, evidence snapshots, methodology context, readiness fingerprints and version-pinned internal review provide a substantially stronger audit trail than a mutable spreadsheet/current-state ledger. The remaining weaknesses are primarily incomplete provenance coverage on rice/legacy export paths and application-level rather than database-level immutability.

AI — Classical AWD ML now uses field-grouped evaluation, managed crop models are clearly exploratory, and the generative-AI workflow is constrained by deterministic server packets and

validation. These are useful assistive features because they remain outside the authoritative accounting boundary.

Verification — On 7 October 2026 the backend suite passed 177 tests with no failures or skips, and the frontend tests, production build, TypeScript check and lint (0 errors) all passed. This supports confidence in software behavior but does not establish scientific, methodology, deployment or external-verification completeness.

14.2 Methodology and Accounting Limitations

VM0051 v1.1 is implemented only for the scoped QA3/default-factor rice pathway; rice leakage, several fertilizer/burning/monitoring/safeguard elements, and independent evidence that remotely detected events satisfy every methodology requirement remain unsupported or partial. VM0042 v2.2 is likewise scoped to supported QA2-style SOC, uncertainty, selected non-SOC/livestock terms and restricted displacement leakage; QA1/process-model quantification, full DSM/sampling, multi-field leakage, mitigation, nonannual reconciliation and complete sequential-period state remain outside the verified scope.

The methodology registry preserves bundles, corrections, applicability and requirement states, but common.additionality is unsupported, blocking and non-overridable in current behavior. Internal fields such as final_issuance or VCU-style values are therefore internal accounting estimates only; Terra-Audit does not implement Verra registry issuance, and internal review is not VVB verification.

14.3 Scientific Validation Limitations

No independent AWD ground-truth dataset has been completed. Sentinel-1 threshold outputs and RF/XGBoost labels therefore measure model-versus-threshold agreement, not field accuracy; orbit/incidence-angle/rainfall effects, revisit gaps and field water-level timing remain unvalidated. Future field work should compare detections with water-level tubes, irrigation logs or partner-verified observations and report event precision/recall, timing error and performance across seasons and locations.

14.4 Provenance and Reproducibility Limitations

Versioned calculation snapshots preserve historical inputs/results and ALM PDF exports use frozen calculation context, but there is no database WORM guarantee, cryptographic full-package signature or single canonical provenance artifact covering every Sentinel scene, geometry revision, detector configuration, attachment hash and executable Git revision. Existing fingerprints are subset-specific change detectors rather than a signed audit manifest; legacy paths may still use current/latest context.

14.5 Security and Operational Limitations

Authentication, server-derived organization scope, roles, token revocation, private artifact access and durable job controls provide a useful foundation, but some project-membership enforcement is inconsistent and the database has no general foreign keys or row-level security. Login throttling, malware/content scanning, deletion cleanup, backup/restore, live worker health, exact deployed revision, production database/storage and PostgreSQL migration were not independently verified. No penetration test or production-security certification is claimed.

14.6 Generative-AI Limitations

Generative outputs are validated for structure, allowlisted references, citation membership, numeric support, corrections and server-owned actions/limitations, but this does not prove semantic entailment or factual truth; output remains a human-review draft. The 20-case evaluation runner exists, yet frozen canonical packet fixtures are incomplete, so no provider-quality benchmark is claimed.

14.7 Prioritized Future Work

P0 — Keep the regression suite green while fixing CI branch/type-generation ordering and endpoint authorization; P0 — resolve the additionality/readiness contradiction without weakening the methodology gate; P1 — make rice provenance calculation-specific and add a tamper-evident evidence manifest; P1 — execute independent AWD ground-truth validation in Bangladesh/South Asia; P1 — verify PostgreSQL/shared-storage/worker/backup/deployment recovery and complete the 20-case AI evaluation; P2 — adopt versioned migrations and continue methodology coverage only where authoritative sources and expert/VVB review support it.

14.8 Conclusion

Terra-Audit demonstrates a credible software architecture for evidence-centered agricultural MRV preparation: it organizes multi-user evidence, executes scoped deterministic VM0051/VM0042 calculations, preserves methodology/readiness and versioned history, supports internal review, and provides audit-oriented analytics/exports. It must not be presented as an autonomous verification or issuance platform; independent AWD validation, complete methodology/program coverage, full provenance sealing, production operational assurance, and external VVB/Verra processes remain separate work.

References

1. Verra, VM0051 v1.1 — Methodology for Improved Management in Rice Production Systems. Repository copy: methodologies/verra/vm0051/VM0051-Improved-Management-in-Rice-Production-Systems-v1.1.pdf.
2. Verra, VM0042 v2.2 — Methodology for Improved Agricultural Land Management. Repository copy: methodologies/verra/vm0042/VM0042v2.2.pdf. The repository also contains the June 2026 correction/clarification file VM0042v2.2_CC_11JUN2026.pdf and VT0014 v1.0 with its correction notice where relevant to the implemented scope.
3. Verra, VMD0054 v1.1 — Methodology for Estimating Leakage from the Displacement of Agricultural Activities. Repository copy: methodologies/verra/vmd0054/VMD0054-v1.1.pdf.
4. IPCC, *2019 Refinement to the 2006 IPCC Guidelines for National Greenhouse Gas Inventories*, Volume 4 (Agriculture, Forestry and Other Land Use), Chapter 5, Table 5.12 (water regime scaling factors) and Table 5.13/5.14 (pre-season and organic-amendment factors); Chapter 10 (livestock enteric fermentation and manure management, cited for the integrated crop-livestock terms in Sections 1.3, 3.2 and 5.2); Chapter 11, Table 11.1 (N₂O emission factors, including pasture/range/paddock-deposited manure N₂O).

5. IPCC, *Fifth Assessment Report (AR5), Climate Change 2013: The Physical Science Basis* — 100-year Global Warming Potential values ($\text{CH}_4 = 28$, $\text{N}_2\text{O} = 265$) used in both carbon estimation engines.
6. Verra, VCS Program Guide; VCS Standard v5.0; and Registration and Issuance Process. Repository copies are stored under methodologies/verra/standards/. These documents are cited only for external program/registration/issuance context; Terra-Audit does not perform registry issuance.
7. European Space Agency / Copernicus Programme, *Sentinel-1 SAR User Guide*. <https://sentinels.copernicus.eu/web/sentinel/user-guides/sentinel-1-sar>
8. Google, *Google Earth Engine API Documentation* (earthengine-api, dataset COPERNICUS/S1_GRD). <https://developers.google.com/earth-engine>
9. R. Lampayan et al., "Adoption and economics of alternate wetting and drying water management for irrigated lowland rice," *Field Crops Research*, vol. 170, pp. 95–108, 2015 (background on AWD practice).
10. A. Savitzky and M. J. E. Golay, "Smoothing and Differentiation of Data by Simplified Least Squares Procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964 (signal smoothing filter used in the data engine).
11. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
12. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794, 2016.
13. A. Bouvet and T. Le Toan, "Use of ENVISAT/ASAR wide-swath data for timely rice fields mapping in the Mekong River Delta," *Remote Sensing of Environment*, vol. 115, no. 4, pp. 1090–1101, 2011.
14. D. B. Nguyen, A. Gruber and W. Wagner, "Mapping rice extent and cropping scheme in the Mekong Delta using Sentinel-1A data," *Remote Sensing Letters*, vol. 7, no. 12, pp. 1209–1218, 2016.
15. R. Torres et al., "GMES Sentinel-1 mission," *Remote Sensing of Environment*, vol. 120, pp. 9–24, 2012.
16. N. Gorelick et al., "Google Earth Engine: Planetary-scale geospatial analysis for everyone," *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.
17. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York: McGraw-Hill Education, 2014.